SKILL EXCHANGE PLATFORM
USING ZOHO CREATOR

A PROJECT REPORT

Submitted by

SWETHA VEERAMANI
(241501261)

in partial fulfilment of the requirements for the course

CB23F35 –
LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF TECHNOLOGY
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



RAJALAKSHMI ENGINEERING COLLEGE
(An Autonomous Institution, Affiliated to Anna University, Chennai)
Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

ACADEMIC YEAR 2026–2027

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled “Skill Exchange Platform Using Zoho Creator” documents the individual project work carried out by SWETHA VEERAMANI (241501261) using the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027.

The application described in this document was configured in Zoho Creator. The implementation details and test observations included in this report shall be updated and verified against the final Zoho Creator application before submission, and any feature whose runtime behaviour has not been independently tested is identified in the report as requiring verification.

The report has been prepared in accordance with the prescribed format of the Department of Artificial Intelligence and Machine Learning, Rajalakshmi Engineering College.

SIGNATURE	**SIGNATURE**
Dr. M. Ayyadurai **(HEAD OF THE DEPARTMENT)** **Department of AIML** **Rajalakshmi Engineering College** **Thandalam – 602 105**	**Dr. J. Jasmine Gabriel** **(PROJECT GUIDE)** **Department of AIML** **Rajalakshmi Engineering College** **Thandalam – 602 105**

Submitted for the End Semester Examination (Individual Project) held on _____.

INTERNAL EXAMINER	**EXTERNAL EXAMINER**

DECLARATION

I, SWETHA VEERAMANI (241501261), hereby declare that this report titled “Skill Exchange Platform Using Zoho Creator” describes the Zoho Creator application developed for the course CB23F35 – Low-Code Application Development using Zoho Creator. The report is intended to describe the application structure, data capture, reports, Deluge scripting and workflow design. Any features whose runtime behaviour has not been independently tested are identified as requiring verification.

I am responsible for reviewing the report, understanding the implementation and demonstrating the application. This declaration should be signed only after reviewing and confirming the contents.

Place: Thandalam, Tamil Nadu **SIGNATURE OF THE STUDENT**

Date: _____ **SWETHA VEERAMANI**

ACKNOWLEDGEMENT

I express my sincere gratitude to the management of Rajalakshmi Engineering College for providing the infrastructure and learning environment required to complete this project. I thank the Principal of Rajalakshmi Engineering College for encouraging students to develop practical applications and to apply the concepts learned in the curriculum.

I extend my sincere thanks to the Head of the Department of Artificial Intelligence and Machine Learning for the constant support and encouragement provided throughout the course. I am deeply grateful to my project guide for the valuable instruction, timely feedback and technical suggestions that shaped the Skill Exchange Platform.

I acknowledge Zoho Corporation for providing the Zoho Creator platform and its documentation, which served as the primary technical reference while configuring forms, lookup fields, Deluge scripts and workflows. I also thank my classmates who reviewed the application during demonstration practice and offered useful observations.

Finally, I thank my family and friends for their patience and support during the preparation of this report.

SWETHA VEERAMANI

ABSTRACT

The Skill Exchange Platform is a Zoho Creator application designed to help students offer skills they already know and request skills they want to learn from other students. Informal skill exchange often depends on separate messages and manual coordination, making it difficult to maintain student profiles, organise learning requests and track the progress of a session. The proposed application brings these activities into a structured low-code system.

The application contains two main forms: Student Skill Profile and Skill Exchange Request. The profile form captures student details, skills offered and wanted, proficiency, availability, learning mode, portfolio information, a certificate upload and a Certificate Summary AI field. The request form links a requester and learning partner through lookup fields and records the skill to learn, skill to teach, session date, duration, status, learning goals, match score and match explanation. A report named Skill_Exchange_Report is intended to provide a consolidated view of exchange requests, supported by a list of all requests and a calendar of sessions.

Three Deluge functions were created: validate_skill_exchange, check_session_date and skill_match_recommendation. A form workflow named skill exchange validation1 was also created, and an email to the requester is generated when a skill exchange request is submitted. The project explores field validation, request tracking and rule-based skill matching. The Certificate Summary AI field is configured on the profile form; its generated summaries, the workflow behaviour and the automatic delivery of emails are to be verified in the live application before they are claimed as working.

The report describes the requirements, design, implementation, testing approach, security considerations and results of the project, and closes with future enhancements. Throughout, each feature is labelled according to the evidence available, so that configured features are distinguished from verified ones.

Keywords: Zoho Creator; Low-Code Development; Skill Exchange; Student Profiles; Deluge; Workflow Automation; Skill Matching.

PROJECT AT A GLANCE

Attribute	Details
Project Title	Skill Exchange Platform Using Zoho Creator
Application Domain	Education / Peer-to-peer learning
Student Name & Register No.	SWETHA VEERAMANI – 241501261
Course	CB23F35 – Low-Code Application Development using Zoho Creator
Platform	Zoho Creator
Application Name	skill exchange platform (short name SEP)
Published / Live URL	Development application; publication to be confirmed before demonstration
Number of Forms / Subforms	2 forms – Student Skill Profile and Skill Exchange Request; no subform recorded
Dataset	Not recorded in the project documentation – name, source and fields to be confirmed in Zoho Creator
Number of Reports	1 report – Skill_Exchange_Report (with a list of all requests and a sessions calendar)
Number of Workflows	1 form workflow – skill exchange validation1; a notification workflow (skill exchange notification) is also recorded
Deluge Functions Created	validate_skill_exchange; check_session_date; skill_match_recommendation
AI Feature	Certificate Summary AI field (configured as Certificate_Keywords) – summarises the certificate uploaded with the student profile
Email Automation	Email generation for the requester when a skill exchange request is submitted
Integrations	No external system integration; email generation is within Zoho Creator
Main Data Relationships	Requester and Learning Partner lookups refer to Student Skill Profile
Roles & Profiles Defined	Student / Requester, Learning Partner and Application Administrator (permissions to be verified)
Primary Purpose	Organise skill offers, learning requests, partner selection and session tracking
Environments Used	Zoho Creator web application (development)
Academic Year	2026–2027

EVIDENCE STATUS LABELS

The following labels are used throughout this report to distinguish what is configured and evidenced from what is documented only and still requires verification.

Label	Meaning
[Configured – Evidence]	The component is configured in the Zoho Creator application and supporting evidence (a screenshot or recorded configuration) is included in this report.
[Configured – Requires Verification]	The component is configured, but its runtime output has not been independently tested; it must be verified in the live application before submission.
[Documented – Requires Verification]	The component is recorded in the project documentation (code, workflow or checklist), but the saved code and runtime behaviour must be confirmed in the Zoho Creator editors.
[Placeholder]	An item (test result, date, signature, screenshot or mark) that is not yet available and is left blank or clearly labelled for completion.

TABLE OF CONTENTS

ABSTRACT ..... **v**
PROJECT AT A GLANCE ..... **vi**
EVIDENCE STATUS LABELS ..... **vii**
LIST OF TABLES ..... **x**
LIST OF FIGURES ..... **xi**
LIST OF ABBREVIATIONS ..... **xii**
CHAPTER 1 INTRODUCTION **1**
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 3
1.6 Organisation of the Report 4
CHAPTER 2 LITERATURE SURVEY AND SYSTEM STUDY **5**
2.1 Study of the Existing System 5
2.2 Limitations of the Existing System 5
2.3 Low-Code versus Traditional Development 6
2.4 Literature Survey 6
2.5 Emerging Trends in Low-Code Development 7
2.6 Proposed System 8
CHAPTER 3 REQUIREMENT ANALYSIS **9**
3.1 Stakeholders and User Roles 9
3.2 Functional Requirements 9
3.3 Non-Functional Requirements 10
3.4 Hardware and Software Requirements 11
3.5 Use Case Model 11
CHAPTER 4 SYSTEM DESIGN **13**
4.1 System Architecture 13
4.2 Data Model 14
4.3 Form Design 15
4.4 Process and Workflow Design 17
4.5 User Interface and Navigation Design 18
4.6 Access Control Design 18
CHAPTER 5 IMPLEMENTATION **20**
5.1 Application Setup 20
5.2 Forms and Data Capture 20
5.3 Dataset Configuration 22
5.4 Reports 22

5.5 Deluge Scripting 24
5.6 Workflow Automation 26
5.7 AI and Analytics Features 27
5.8 Application Screens 28
CHAPTER 6 TESTING AND DEBUGGING 30
6.1 Testing Strategy 30
6.2 Test Cases 30
6.3 Debugging Deluge Scripts 32
6.4 Defect Log 32
6.5 User Acceptance Testing 33
CHAPTER 7 DEPLOYMENT, SECURITY AND GOVERNANCE 34
7.1 Environments and Deployment Strategy 34
7.2 Performance Optimisation 35
7.3 Security Features 35
7.4 Governance and Administration 36
CHAPTER 8 RESULTS AND DISCUSSION 37
8.1 Application Output 37
8.2 Achievement of Objectives 38
8.3 Comparison with the Existing System 39
8.4 Limitations of the Low-Code Approach 40
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 40
CHAPTER 9 CONCLUSION AND FUTURE ENHANCEMENTS 41
9.1 Conclusion 41
9.2 Future Enhancements 41
9.3 Learning Outcomes and Career Relevance 42
REFERENCES 44
APPENDIX A – FEATURE COVERAGE CHECKLIST 45
APPENDIX B – COURSE OUTCOME MAPPING 46
APPENDIX C – COMPLETE DELUGE SOURCE CODE 47
C.1 validate_skill_exchange 47
C.2 check_session_date 47
C.3 skill_match_recommendation 47
APPENDIX D – USER MANUAL 48
D.1 Accessing the Application 48
D.2 Add a Student Profile 48
D.3 Create a Skill Exchange Request 48
D.4 Review and Update a Request 49
APPENDIX E – EVALUATION RUBRIC 50
APPENDIX F – VIVA-VOCE RECORD SHEET 52

LIST OF TABLES

Table 2.1: Conventional Development versus Low-Code Development for this Project 6
Table 2.2: Summary of Literature Survey 6
Table 2.3: Limitations of the Existing Process and the Corresponding Features of the Proposed System 8
Table 3.1: Stakeholders and User Roles 9
Table 3.2: Functional Requirements 9
Table 3.3: Non-Functional Requirements 10
Table 3.4: Hardware and Software Requirements 11
Table 3.5: Use Case Description – Create a Student Profile 11
Table 3.6: Use Case Description – Submit a Skill Exchange Request 11
Table 3.7: Use Case Description – Review and Update a Request 12
Table 4.1: Entities and Relationships 14
Table 4.2: Field Specification – Student Skill Profile Form 15
Table 4.3: Field Specification – Skill Exchange Request Form 16
Table 4.4: Request Status Values and Their Meaning 17
Table 4.5: Summary of Workflow and Function Design 18
Table 4.6: Role–Permission Matrix 18
Table 5.1: Dataset Configuration Attributes 22
Table 5.2: Reports Configured 22
Table 5.3: Sample Request Records Shown in Figure 5.4 23
Table 5.4: Deluge Functions Implemented 24
Table 5.5: Workflow Configuration 27
Table 5.6: Index of Application Screenshots 29
Table 6.1: Test Cases 31
Table 6.2: Defect Log 33
Table 6.3: User Acceptance Feedback 33
Table 7.1: Deployment Checklist 34
Table 7.2: Data Items and Their Sensitivity 35
Table 7.3: Risks and Mitigation 36
Table 8.1: Mapping of Objectives to Outcomes 38
Table 8.2: Informal Process versus the Skill Exchange Platform 39
Table A.1: Feature Coverage Checklist 45
Table B.1: Course Outcome Mapping 46
Table E.1: Evaluation Rubric for the Individual Project 50
Table E.2: Performance Level Descriptors 50
Table F.1: Viva-Voce Questions and Responses 52

Table F.2: Marks Summary 52

LIST OF FIGURES

Figure 4.1: Skill Exchange Platform Architecture 13
Figure 4.2: Student Profile, Exchange Request and Report Relationships 15
Figure 5.1: Student Skill Profile form – personal details and skills (live mode) 20
Figure 5.2: Student Skill Profile form – availability, learning mode and certificate upload 21
Figure 5.3: Skill Exchange Request form 21
Figure 5.4: All Skill Exchange Requests report 23
Figure 5.5: Skill_Exchange_Report showing request status 24
Figure 5.6: Calendar view of scheduled sessions 24
Figure 5.7: Workflow “skill exchange validation1” – Deluge validation scripts 26
Figure 5.8: Workflow “skill exchange notification” – email on accepted requests 28
Figure 8.1: Consolidated report of exchange requests (result view; see also Figure 5.5) 37
Figure 8.2: Calendar of scheduled sessions (result view; see also Figure 5.6) 38

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
AIML	Artificial Intelligence and Machine Learning
Deluge	Data Enriched Language for the Universal Grid Environment
ER	Entity–Relationship
FR	Functional Requirement
ID	Identifier
NFR	Non-Functional Requirement
RBAC	Role-Based Access Control
TC	Test Case
URL	Uniform Resource Locator
Zoho Creator	Low-code application development platform

CHAPTER 1
INTRODUCTION

1.1 Background

Peer-to-peer learning is one of the oldest and most effective ways in which students acquire practical skills. In every classroom, laboratory and hostel, students possess a wide range of strengths that are rarely visible beyond their immediate circle of friends. One student may be comfortable with Python programming, another with Java, a third with presentation design, spreadsheet analysis or public speaking. At the same time, each of these students also has areas in which they want to improve, and those areas frequently coincide with the strengths of their classmates.

A skill exchange is a reciprocal arrangement built on this observation: one student teaches what they know and, in return, learns what they want from the other. Because both participants contribute and both benefit, the arrangement is motivating, costs nothing and strengthens teamwork. Teaching also reinforces the understanding of the teacher, since explaining a concept to another person exposes gaps that silent study can hide. For students of engineering programmes, where practical competence in tools and programming languages is as important as theoretical knowledge, such exchanges can supplement classroom instruction in a very direct way.

In practice, however, informal skill exchange depends on chance. A student must know whom to ask, must find the person at a convenient time and must remember what was agreed. Offers and requests are scattered across messaging groups, notebooks and spreadsheets, and there is no shared record of who can teach what, who is waiting to learn what, and which sessions have actually taken place. A structured system that records skills offered, skills wanted, availability, requests and session outcomes in one place can remove most of this friction.

This project, the **Skill Exchange Platform**, implements such a system using Zoho Creator, a low-code application development platform. The application was developed as the individual project for the course CB23F35 – Low-Code Application Development using Zoho Creator. It demonstrates how forms, lookup relationships, reports, calendar views, Deluge scripts and workflows can be combined to model a real-world process without building a separate database or web interface from scratch.

1.2 Problem Statement

Without a central system, students rely on informal chats or spreadsheets to find learning partners. Skill information becomes outdated, requests may be missed, and there is no consistent way to track a request from the moment it is raised until the session is completed. A low-code application is proposed to centralise student profiles and exchange requests so that every stage of the exchange is recorded in a structured, searchable and reviewable form.

A study of how students currently arrange peer learning reveals the following recurring problems, which together define the problem that the application addresses:

- **Scattered information:** details of who can teach which skill are held in personal memory and in group messages rather than in a shared directory.
- **No structured matching:** a student searching for a partner cannot easily compare the skills they want with the skills that others offer.
- **Missed or forgotten requests:** a request made in a busy chat conversation can be overlooked, leaving the requester without a reply.
- **Inconsistent session details:** the date, duration and learning goals of a session are agreed verbally and are rarely recorded in a uniform way.
- **No status tracking:** nobody can tell, without asking, whether a request is pending, accepted, rejected or completed.
- **Invalid arrangements:** nothing prevents a request from being raised for a date that has already passed, or from naming the same student as both requester and partner.

1.3 Objectives

The overall aim of the project is to design and configure a Zoho Creator application that organises peer skill exchange. The aim is broken down into the following specific objectives, each of which is later traced to an implementation component in Table 8.1:

- **Create student profiles with skills offered and skills wanted.** The profile form acts as a directory so that any student can see which skills are available and which are in demand.
- **Record skill exchange requests between two student profiles.** A request links a requester and a learning partner through lookup fields rather than free text, so that both participants are always valid records.
- **Capture the skill to learn, skill to teach, date, duration and learning goals.** These fields describe the exchange fully enough for both students to prepare for the session.
- **Track request status using Pending, Accepted, Rejected and Completed values.** A single status field gives every request a visible position in its life cycle.
- **Provide a report for reviewing exchange requests.** A consolidated report and a calendar view allow requests and sessions to be reviewed without opening each record.
- **Use Deluge scripts and a workflow to support validation and matching logic.** Scripts prevent invalid submissions and provide a place for rule-based matching.

- **Use the Certificate Summary AI field and generate an email to the requester.** The profile form carries a certificate upload with an AI summary field, and a request submission triggers an email, without claiming any output that has not been verified.

1.4 Scope of the Project

The scope of the project covers two forms, a report, three named Deluge functions and a form workflow. The intended use is student-to-student skill exchange within an institution. The project concentrates on correct data modelling, input validation and clear tracking of requests, because these are the foundations on which more advanced features can later be built.

1.4.1 In Scope

- The Student Skill Profile form with personal details, skills, proficiency, availability, learning mode, portfolio URL, certificate upload and Certificate Summary AI field.
- The Skill Exchange Request form with requester and learning partner lookups, skills, session date and duration, request status, learning goals and match fields.
- The Skill_Exchange_Report, the list view of all requests and a calendar view of scheduled sessions.
- Three Deluge functions, one validation workflow and one notification workflow.

1.4.2 Out of Scope

- Payment, grading or certification of sessions; the platform records exchanges but does not assess them.
- Video conferencing or chat inside the application; sessions are conducted outside the platform.
- Integration with institutional systems such as the student information system or the learning management system.
- A published production deployment with real student data; the report describes a development-stage application.

Live notification delivery, automatic partner recommendation and AI-generated certificate summaries should be considered verified only after successful tests have been recorded. The report therefore labels every feature with an evidence status, as explained in the section on evidence status labels at the beginning of this document.

1.5 Justification for a Low-Code Approach

Zoho Creator provides visual form design, lookup fields, reports, workflow configuration and Deluge scripting in one environment. This makes it suitable for a

student project that demonstrates data modelling and automation without building a separate database and web interface from scratch [1]–[3]. In a conventional approach the same application would require the design of database tables, a server-side programme to read and write them, a user interface layer, an authentication mechanism and a hosting arrangement. In a low-code approach, the platform supplies each of these layers and the developer concentrates on the structure and rules of the application. Table 2.1 in Chapter 2 contrasts the two approaches for the activities involved in this project.

The main benefit for this project is speed and focus. Because the platform handles hosting, storage and the user interface, development effort can be spent on understanding the process, modelling the data correctly and testing the rules. The principal trade-off is that the developer works within the capabilities and conventions of the platform, for example in the exact naming of field link names used by Deluge scripts. This trade-off is acceptable for an application of this size.

1.6 Organisation of the Report

The report is organised into nine chapters followed by references and appendices. Chapter 1 introduces the project, its problem statement and objectives. Chapter 2 studies the existing process, reviews the low-code approach and presents the proposed system. Chapter 3 records the requirements, including stakeholders, functional and non-functional requirements and use cases. Chapter 4 presents the architecture, data model, form design, workflow design and access control design. Chapter 5 describes the Zoho Creator implementation, with screenshots placed beside the component they illustrate. Chapter 6 defines the testing approach, test cases and defect log. Chapter 7 covers deployment, security and governance. Chapter 8 reviews results and discussion, and Chapter 9 concludes the report with future enhancements. The appendices contain the feature coverage checklist, course outcome mapping, Deluge source code, user manual, evaluation rubric and viva-voce record sheet.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

In an informal skill-sharing process, students may ask classmates or use group chats to find someone who can teach a skill. The information about available skills, schedules and request status is spread across conversations and is difficult to review in one place. The process works for small groups of friends but does not scale to a class, a department or an institution, where the number of students and skills is too large to be held in anyone's memory.

2.1.1 Channels Commonly Used

- **Messaging groups:** a student posts a message such as "Can anyone teach me Java?" and waits for a reply. Responses are mixed with unrelated conversation and may arrive days later.
- **Personal contacts:** students approach those they already know. This limits the pool of possible partners to the student's own friend circle.
- **Shared spreadsheets or notes:** a few groups maintain a list of skills, but the list is updated by hand, is rarely complete and quickly becomes outdated.
- **Notice boards and announcements:** these reach many students but do not capture who responded or what was agreed.

2.1.2 Typical Sequence of an Informal Exchange

A typical exchange in the existing process follows the sequence below. Every step depends on manual communication, and none of the steps leaves a structured record.

1. A student realises they want to learn a skill and posts a request in a chat group.
2. A classmate replies that they can help and asks what the student can teach in return.
3. The two students negotiate a date and duration through several messages.
4. One of the students forgets or postpones the session; the other may not be informed.
5. If the session takes place, nobody records that it was completed, so the exchange cannot be reviewed later.

2.2 Limitations of the Existing System

The informal process has the following limitations. Each one corresponds to a feature of the proposed system, as shown in Table 2.3.

- **No single, structured student skill directory.** Skill information is held by individuals, so a student cannot browse the skills available in the class.

- **Difficulty comparing skills offered with skills wanted.** Matching is done mentally, and a suitable partner is often missed simply because the two students never met.
- **Learning requests can be overlooked in chat messages.** A request is only as visible as the last message in the conversation.
- **Session dates, duration and learning goals may not be recorded consistently.** Each pair of students agrees on details in a different way, so no two sessions are described in the same format.
- **No consolidated report of pending, accepted or completed exchanges.** Coordinators and faculty mentors cannot see how much peer learning is taking place.

2.3 Low-Code versus Traditional Development

The comparison below summarises why a low-code platform was preferred for this project.

Table 2.1: Conventional Development versus Low-Code Development for this Project

Activity	Conventional approach	Zoho Creator approach
Data storage	Design tables, keys and relationships in a separate database.	Each form is a table; lookup fields create relationships automatically.
User interface	Write pages and form-handling code.	Drag-and-drop form builder with field types and layouts.
Validation	Write server and client validation code.	Field properties plus short Deluge scripts on submission.
Reports	Write queries and display pages.	Configure list, calendar and other report types visually.
Automation	Write background jobs and mail code.	Form workflows with condition, action and send-mail blocks.
Hosting and access	Provision servers and authentication.	Provided by the platform; access through a browser.

2.4 Literature Survey

The primary technical references for this project are the Zoho Creator product documentation, the Deluge scripting documentation and the Zoho Creator help pages on forms, reports and workflows [1]–[3]. These platform concepts are used directly to design the profile and request records, and the report cites the documentation sections that were consulted.

Table 2.2: Summary of Literature Survey

Source	Topic Studied	Relevance
[1] Zoho Creator product documentation	Platform capabilities: forms, reports, pages and workflows	Used to design the Student Skill Profile and Skill Exchange Request forms and the reports.
[2] Zoho Creator Deluge scripting documentation	Deluge syntax, form events and functions	Used for the validation and matching functions.
[3] Zoho Creator forms, reports and workflow documentation	Form workflows, actions and conditions	Used for the validation and notification workflows.

2.4.1 Peer Learning and Skill Exchange

Peer learning describes educational practices in which students learn with and from each other without a teacher as the central source of knowledge. Skill exchange is a reciprocal form of peer learning in which each participant acts as both teacher and learner. The concept is widely used in community learning initiatives and student clubs. The application studied here does not attempt to measure learning effectiveness; its contribution is to provide the organisational structure that makes such exchanges easier to arrange and track.

2.4.2 Low-Code Application Platforms

Low-code platforms allow applications to be assembled mainly through visual tools, with a small amount of code used where the visual tools are not sufficient. Zoho Creator is one such platform; it offers form builders, report builders, page builders, workflows and a scripting language called Deluge [1]. The platform stores data in its own cloud-hosted database and exposes it to the application through forms and reports, so the developer never writes database definition statements.

2.4.3 Deluge Scripting

Deluge, the Data Enriched Language for the Universal Grid Environment, is the scripting language of Zoho products [2]. In Zoho Creator, Deluge scripts can be attached to form events such as load, validation on submit and successful submission, and can also be saved as reusable functions. Typical statements read the values of form fields through the input object, display messages with alert, and stop a submission with cancel submit. The three functions in this project use exactly these constructs.

2.4.4 Workflows and Automation

A form workflow in Zoho Creator is a set of actions that run when a defined event occurs on a form, optionally subject to a condition [3]. Actions include running a Deluge script, sending an email, creating or updating records and displaying messages. Workflows are the mechanism through which this project applies validation on submission and sends an email to the requester.

2.5 Emerging Trends in Low-Code Development

Low-code platforms are widely used by organisations that need business applications quickly, and they increasingly allow AI services to be embedded in ordinary forms. The Certificate Summary AI field used in this project is an example: it takes an uploaded certificate as input and stores a generated summary in the form, without any separate AI application being built. Platform features of this kind are treated in this report as configured features whose output must be verified before they are claimed as working.

The developer must still understand the data model, the field link names and the validation rules, because the platform executes the logic exactly as it is configured. A related point concerns recommendations: a matching rule written by hand is rule-based matching rather than artificial intelligence, and the report describes it as such. The matching function of this project points to how data-driven recommendations could later be added, as discussed in Chapter 9.

2.6 Proposed System

The proposed Skill Exchange Platform stores student profiles and exchange requests in related forms. Lookup fields connect the requester and learning partner to student profile records. The report provides a central view of requests, while Deluge functions and a workflow provide a place to implement validation and matching behaviour. The profile form additionally carries a certificate upload and a Certificate Summary AI field, and a request triggers the generation of an email to the requester.

Table 2.3: Limitations of the Existing Process and the Corresponding Features of the Proposed System

Limitation of existing process	Feature of the proposed system
No structured skill directory	Student Skill Profile form with skills offered and skills wanted.
Hard to compare offers and wants	Lookup-based requests and the skill matching function.
Requests overlooked in chats	Skill Exchange Request form and a report listing every request.
Inconsistent session details	Mandatory structured fields for date, duration and learning goals.
No status visibility	Request Status field with Pending, Accepted, Rejected and Completed.
Invalid arrangements possible	Deluge validation for past dates and self-matching.
No acknowledgement to the requester	Email generation when a request is submitted.

2.6.1 Design Principles

Three principles guided the design of the proposed system. The first is **single source of truth**: every student and every request exists once, as a record, and all views are derived from those records. The second is **validation at the point of entry**: it is easier to

prevent an invalid request than to correct it later, so the rules are applied when the form is submitted. The third is **honest verification**: the report does not claim that a feature works until it has been demonstrated, and so it distinguishes configured features from verified ones.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

A stakeholder is any person or group that uses the application or is affected by it. Three stakeholders were identified; their responsibilities are listed in Table 3.1.

Table 3.1: Stakeholders and User Roles

Role	Responsibilities
Student / Requester	Create a profile, choose a learning partner and submit a request.
Learning Partner	Review a request and coordinate the learning session.
Application Administrator	Maintain forms, review records and manage configuration.

The **student or requester** is the primary user. This person maintains an accurate profile, decides which skill to learn and which to offer in return, and submits a request naming a partner. The **learning partner** is also a student, and the same person can act as requester in one exchange and partner in another. The partner reviews the request and, by agreeing on the details, moves the request towards acceptance. The **application administrator** is typically the project owner or a faculty coordinator, who maintains the forms and workflows, reviews the records for quality and is responsible for the permissions and privacy of the data.

3.2 Functional Requirements

Functional requirements describe what the application must do. Nine functional requirements were defined and given a priority of High or Medium, as shown in Table 3.2.

Table 3.2: Functional Requirements

ID	Functional Requirement	Priority
FR-01	Capture student name, email, department and profile details.	High
FR-02	Record skills offered, skills wanted and proficiency level.	High
FR-03	Record availability, learning mode and portfolio URL.	Medium
FR-04	Create an exchange request linked to requester and learning partner.	High
FR-05	Record skills to learn/teach, session date, duration and goals.	High
FR-06	Track request status: Pending, Accepted, Rejected or Completed.	High
FR-07	Display request records in Skill_Exchange_Report.	High
FR-08	Use Deluge functions for validation and skill matching logic.	Medium
FR-09	Use the Certificate Summary AI field and email generation for requests; verify their outputs.	Medium

- **FR-01 and FR-02 (profile identity and skills)** make the profile form a skill directory. Without accurate names, contact details and skills, no other function is meaningful, hence the High priority.
- **FR-03 (availability, mode and portfolio)** supports practical arrangements: the days on which a student is free, whether they prefer online or offline learning, and an optional link to prior work. It is rated Medium because an exchange can still be arranged without it.
- **FR-04 and FR-05 (request creation)** define the core transaction of the application. A request must reference two valid student records and must describe the exchange completely.
- **FR-06 (status tracking)** gives each request a position in its life cycle and is the basis for reporting.
- **FR-07 (report)** ensures that the stored requests can be reviewed together.
- **FR-08 (Deluge logic)** provides validation and matching rules that cannot be expressed through field properties alone.
- **FR-09 (AI summary and email)** adds convenience features, and carries the explicit condition that their outputs must be verified before being claimed.

3.3 Non-Functional Requirements

Non-functional requirements describe the qualities of the application rather than its features. Six categories were considered, as listed in Table 3.3.

Table 3.3: Non-Functional Requirements

Category	Requirement
Usability	Forms should use clear labels and simple selection fields.
Data consistency	Lookup fields should reference valid student profile records.
Validation	Past session dates and invalid self-matching should be prevented if the rules are configured.
Maintainability	Use consistent field link names in Deluge scripts.
Reliability	Workflow and script behaviour must be tested with valid and invalid sample records.
Privacy	Student contact and profile data should only be available to authorised users.

Usability is addressed by using drop-down lists, check boxes and date pickers in place of free text wherever the set of valid values is known, as seen in the form screenshots in Chapter 5. **Data consistency** is achieved through lookup fields, which accept only existing profile records and so prevent misspelt or non-existent partner names. **Validation** requirements are expressed conditionally ("if the rules are configured") because their enforcement depends on the workflow being enabled and correct, which is a matter for testing. **Maintainability** is a particular concern for Deluge: a script that

uses a field name different from the field link name shown in the form builder will fail, and this exact problem was encountered during development (Section 6.3 and Table 6.2). **Reliability** is supported by the test plan in Chapter 6. **Privacy** is important because the application stores names, email addresses and uploaded certificates, which are covered in Chapter 7.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator (cloud-hosted low-code platform).
Scripting language	Deluge, used inside Creator for functions and workflows.
Browser	A current web browser such as Chrome, Edge or Firefox.
Account	A Zoho Creator account with permission to create and edit applications.
Network	An internet connection; the application is accessed online.
Hardware	A computer or mobile device capable of running a browser; no further hardware is required.
Database / server	Not required separately; provided by the platform.

Because Zoho Creator is delivered as a cloud service, there are no hardware requirements beyond a computer or mobile device capable of running a browser. This is a significant advantage for a student project, since the application can be developed, demonstrated and reviewed from any location. The project does not require a separately hosted database or custom server for its core form-and-report structure.

3.5 Use Case Model

Three principal use cases describe how the stakeholders interact with the application. The main use case is the submission of a skill exchange request (UC-02), which depends on the profiles created in UC-01 and is followed by the review described in UC-03.

Table 3.5: Use Case Description – Create a Student Profile

Item	Description
Use Case ID	UC-01
Use Case Name	Create a Student Profile
Primary Actor	Student
Pre-condition	The student has access to the application.
Main Flow	1. Student opens the Student Skill Profile form. 2. Enters name, email and department. 3. Selects the skills offered and wanted. 4. Chooses a proficiency level, ticks the available days and selects a learning mode. 5. Optionally uploads a certificate and adds a portfolio URL. 6. Submits the form.
Alternate Flow	None recorded in the project documentation.

Item	Description
Post-condition	A profile record is stored and becomes available in the requester and learning partner lookups.

Table 3.6: Use Case Description – Submit a Skill Exchange Request

Item	Description
Use Case ID	UC-02
Use Case Name	Submit a Skill Exchange Request
Primary Actor	Student (requester)
Pre-condition	At least two student profiles exist.
Main Flow	1. Requester opens the Skill Exchange Request form. 2. Selects themselves as requester. 3. Chooses a different student as learning partner. 4. Selects the skill to learn and the skill to teach. 5. Picks a session date and duration, sets the request status and writes the learning goals. 6. Submits the form.
Alternate Flow	If the session date is in the past, or the requester and partner are the same student, the validation scripts display a message and cancel the submission.
Post-condition	A request record is stored, appears in the report and triggers the configured email.

Table 3.7: Use Case Description – Review and Update a Request

Item	Description
Use Case ID	UC-03
Use Case Name	Review and Update a Request
Primary Actor	Learning partner or administrator
Pre-condition	A request exists.
Main Flow	1. Actor opens the report or calendar view. 2. Selects a request and reviews the details. 3. Updates the Request Status to Accepted, Rejected or Completed as the exchange progresses.
Alternate Flow	None recorded in the project documentation.
Post-condition	The updated status is visible in the report, and an accepted request appears in the calendar on its session date.

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

The user interacts with Zoho Creator forms and reports through a browser. Zoho Creator stores form records, resolves lookup relationships, displays reports and runs configured workflow actions and Deluge logic. The flow of data is: Student → Student Skill Profile → Skill Exchange Request → Skill_Exchange_Report, with Deluge validation / matching logic supporting the request.

4.1.1 Architectural Layers

Figure 4.1 shows the architecture as three cooperating layers. The topmost layer contains the **users**: the student or requester, the learning partner and the application administrator. All of them reach the application through the same entry point, a **web browser** that opens the Zoho Creator web interface. The lowest and largest layer is the **Zoho Creator Skill Exchange Platform** itself, which contains the forms, the report, the lookups, the Deluge functions and the form workflow.

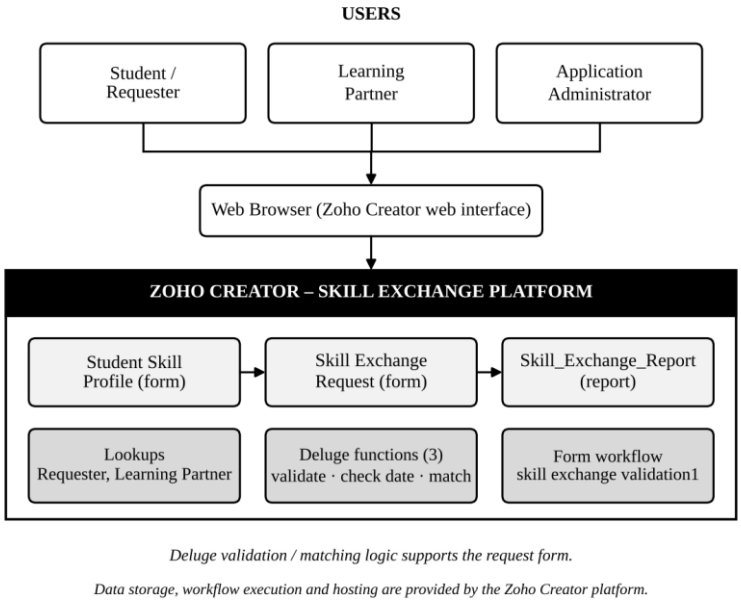


Figure 4.1: Skill Exchange Platform Architecture

4.1.2 Components Inside the Platform

- **Student Skill Profile (form)** stores the identity, skills and availability of every student.
- **Skill Exchange Request (form)** stores each exchange and refers to two profile records.
- **Skill_Exchange_Report (report)** presents the stored requests in a consolidated view.

- **Lookups** (Requester and Learning Partner) are the links through which the request form refers to the profile form.
- **Deluge functions (three)** hold the validation and matching logic: validate, check date and match.
- **Form workflow** (skill exchange validation1) runs the validation scripts when a request is submitted.

4.1.3 Data Flow

The data flow through the architecture can be followed in five steps. First, a student enters profile data, which Zoho Creator stores as a profile record. Second, a requester opens the request form; the lookup fields read their lists of options from the stored profile records. Third, on submission the workflow runs the Deluge validation scripts, which either allow the submission or cancel it with a message. Fourth, an accepted submission is stored as a request record, and the match logic and email actions may run. Fifth, the report and calendar read the stored request records and present them to users. Data storage, workflow execution and hosting are all provided by the Zoho Creator platform, so no component outside the platform is needed.

4.2 Data Model

The data model consists of two stored entities, the student profile and the exchange request, and one derived entity, the report. Table 4.1 summarises their purpose and relationships.

Table 4.1: Entities and Relationships

Entity / Form	Purpose	Relationship
Student Skill Profile	Stores identity, department, skills offered/wanted, proficiency, availability, learning mode, portfolio URL and certificate upload.	Referenced by both requester and learning partner lookups.
Skill Exchange Request	Stores the requested exchange, session details, status, goals and match fields.	Contains Requester and Learning Partner lookup fields.
Skill_Exchange_Report	Provides a consolidated view of exchange requests.	Based on Skill Exchange Request records.

4.2.1 Entity-Relationship View

Figure 4.2 shows the relationships among the three entities. The relationship between a profile and a request is **one-to-many**, marked 1 – N in the diagram: one student profile can be referred to by many requests, but each lookup field in a request refers to exactly one profile. Because a request has two lookup fields, the same profile table is referred to twice, once in the role of **Requester** and once in the role of **Learning Partner**. The

report is not a separate store of data; it is drawn from the request records, as the arrow labelled “report is based on request records” indicates.

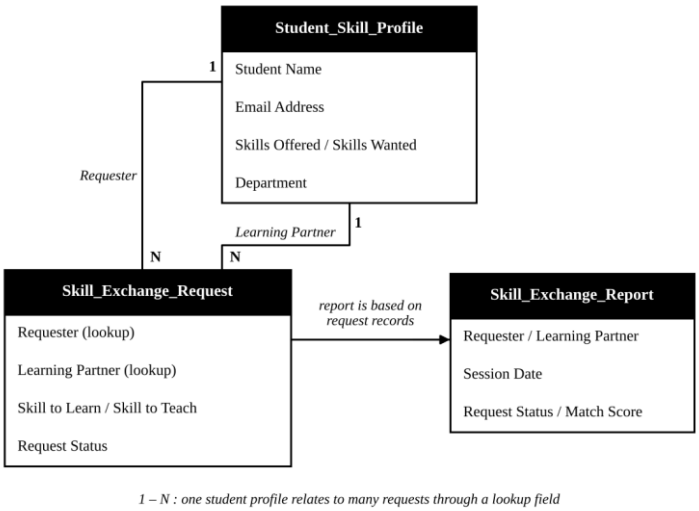


Figure 4.2: Student Profile, Exchange Request and Report Relationships

4.2.2 Behaviour of Lookup Relationships

A lookup field displays values from another form in a drop-down list and stores a reference to the selected record. This has three practical consequences for the application. First, a requester cannot type a name that does not exist, which protects data consistency. Second, if the partner’s profile is later corrected, the corrected name appears wherever the request is displayed, because the request stores a reference rather than a copy. Third, the two lookups in the request form point to the same profile table and therefore offer the same list, which is why a validation rule is needed to stop a student from selecting themselves in both roles.

4.3 Form Design

Each form was designed by listing the data needed to satisfy the functional requirements and choosing the field type that best constrains the value. Choice fields, check boxes, date fields and lookups were preferred to free text so that the stored data is uniform. The final field link names should be verified in the Zoho Creator builder before they are used in Deluge scripts, because a mismatch between a link name and the script is a common cause of failure.

4.3.1 Student Skill Profile

Table 4.2: Field Specification – Student Skill Profile Form

Field	Purpose / Type
Student Name	Identifies the student.
Email Address	Contact information.
Department	Academic department.
Skills Offered	Skills the student can teach.

Field	Purpose / Type
Skills Wanted	Skills the student wants to learn.
Proficiency Level	Indicates skill experience.
Available Days	Availability for sessions.
Learning Mode	Preferred learning format.
Portfolio URL	Optional link to work or projects.
Certificate Upload	Upload of the student's certificate (Figure 5.2).
Certificate Summary AI Field	AI field that summarises the uploaded certificate; configured as Certificate_Keywords (Section 5.7.1, Table A.1).

The field design follows the way a student would describe themselves to a prospective partner. **Student Name** and **Email Address** identify and allow contact with the student; the name is split into first and last name so that records can be sorted and displayed neatly. **Department** is a selection field so that departments are spelt uniformly. **Skills Offered** and **Skills Wanted** are the heart of the profile, since they are what a requester compares. **Proficiency Level** indicates how confident the student is in the offered skill, which helps a learner set expectations. **Available Days** is a set of check boxes from Sunday to Saturday, allowing several days to be chosen. **Learning Mode** records whether the student prefers, for example, an online or in-person session. **Portfolio URL** is optional and gives a prospective partner evidence of prior work. **Certificate Upload** and the **Certificate Summary AI Field** provide supporting evidence of a skill and a short automatic description of it.

4.3.2 Skill Exchange Request

Table 4.3: Field Specification – Skill Exchange Request Form

Field	Purpose / Type
Requester	Lookup to Student Skill Profile.
Learning Partner	Lookup to Student Skill Profile.
Skill to Learn	Skill requested by the requester.
Skill to Teach	Skill offered in exchange.
Session Date	Planned date for the session.
Session Duration	Expected session length.
Request Status	Pending, Accepted, Rejected or Completed.
Learning Goals	Description of intended learning outcomes.
AI Match Explanation	Multiline field intended to explain a match; currently described as text unless verified otherwise.

Field link names used by the Deluge functions (Appendix C) are Requester, Learning_Partner, Session_Date, Skill_to_Learn, Skill_to_Teach and Match_Score; they must match the form builder exactly. A field link name is the internal identifier

through which scripts read a field, and it is distinct from the label shown to users. A mismatch, such as writing Session_Date in a script when the form builder has defined session_date, causes the script to fail, which is a recurring source of errors in Deluge development and is discussed in Section 6.3.

4.3.3 Request Status Life Cycle

The Request Status field gives every request a position in a simple life cycle. A new request is normally **Pending** until the learning partner responds. It then becomes **Accepted** if the partner agrees, or **Rejected** if the partner declines. An accepted request becomes **Completed** after the session has taken place. Table 4.4 describes the meaning of each value and the typical action that causes the change.

Table 4.4: Request Status Values and Their Meaning

Status	Meaning	Typical next action
Pending	The request has been submitted and awaits a response.	Learning partner reviews the request.
Accepted	The partner has agreed to the exchange and the session is scheduled.	Session takes place on the session date.
Rejected	The partner has declined or cannot attend.	Requester may raise a new request.
Completed	The session has taken place.	Record is retained for review.

4.4 Process and Workflow Design

A form workflow named skill exchange validation1 was created for skill exchange validation. The intended validation rules are to ensure the session date is today or a future date and to prevent the requester from selecting themselves as the learning partner. The exact configured actions and successful runtime behaviour should be confirmed in the Creator workflow editor before the report is submitted. [Documented – Requires Verification]

4.4.1 Validation Rules

Two validation rules were designed. The first is a **date rule**: a session must be planned for today or a later date, because a session in the past cannot be arranged. The second is a **self-match rule**: the requester and the learning partner must be different students, because a student cannot exchange skills with themselves. Both rules are evaluated when the form is submitted, and in either case a violation displays a message to the user and cancels the submission so that no invalid record is stored.

4.4.2 Notification Design

A second workflow, skill exchange notification, was designed to send an email when a request is saved. It runs on successful form submission, which means it executes only after the record has been stored, and it is subject to a condition on the request status.

This arrangement ensures that no email is generated for a submission that was cancelled by a validation rule.

4.4.3 Matching Logic Design

A matching function was planned to compare the skills a requester wants with the skills a partner offers and to store a score in the Match Score field, accompanied by a short explanation. The rule-based version recorded in Appendix C.3 is deliberately simple; Chapter 9 describes how it can be extended. If the application uses a manually written rule, that feature is rule-based matching rather than artificial intelligence, and the report describes it as such.

Table 4.5: Summary of Workflow and Function Design

Component	Event / trigger	Intended action
validate_skill_exchange	Form submission	Block a request whose requester and partner are the same student.
check_session_date	Form submission	Block a request whose session date is in the past.
skill_match_recommendation	Form submission	Compute a match score and store it in Match Score.
skill exchange validation1	Validation on submission	Run the validation scripts before the record is saved.
skill exchange notification	Successful form submission	Send an email when the request status condition is met.

4.5 User Interface and Navigation Design

The application is named “skill exchange platform”, and the screenshots show its short name SEP at the top left of the navigation bar. The navigation bar has two menus, **Student Skill Profiles** and **Skill Exchange Requests**. The first menu contains the profile form and its records; the second contains the request form, the list of all requests, the report and the calendar of sessions. This organisation mirrors the two main entities of the data model and lets a user move directly to the part of the application they need.

Students use the profile form to describe their skills and the request form to propose an exchange; the administrator and the learning partner use the report and the calendar to review requests. Choice fields, check boxes, date pickers and lookups are used wherever the valid values are known, so that users select rather than type.

4.6 Access Control Design

Table 4.6: Role–Permission Matrix

Component	Administrator	Student / Requester	Status
Student Skill Profile form	Review all profiles; maintain the form	Create and update own profile; view the skill directory	Verify in application

Component	Administrator	Student / Requester	Status
Skill Exchange Request form	Review all requests	Submit a request; the learning partner reviews it	Verify in application
Report and calendar	View all requests and sessions	Session details visible to the two participants	Verify in application
Certificate upload and summary	View	Owner only	Verify in application
Forms, scripts and workflows	Administrative access	No access	Platform permission setting

The permission matrix describes the intended access model, which follows the least-privilege principle described in Section 7.3. The actual permissions must be checked in Zoho Creator using separate administrator and student accounts before the report states that role-based access has been tested. Role-based access and multi-user testing are treated as a future enhancement in Section 9.2.6.

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application was created in Zoho Creator using visual form and report builders. The project is organised around the student profile form, the exchange request form, a consolidated report, Deluge functions and a form workflow. The two main forms are Student Skill Profile and Skill Exchange Request. The forms are arranged so that student profiles are created before requests are submitted, and the Requester and Learning Partner lookups in the request form refer to the profile form. The navigation structure of the application is described in Section 4.5.

5.2 Forms and Data Capture

The Student Skill Profile form acts as the student directory. The Skill Exchange Request form records an exchange between two profiles and stores the operational details needed to coordinate a session. Lookup fields reduce repeated typing and connect the request to profile records. The profile form also includes a certificate upload field and a Certificate Summary AI field that summarises the uploaded certificate (Figure 5.2, Table A.1).

5.2.1 Student Skill Profile Form – Personal Details and Skills

The upper part of the profile form, shown in Figure 5.1 in live mode, captures the personal and skill details of a student. The student name is entered in two boxes labelled First Name and Last Name, followed by the email address. The department and proficiency level are chosen from drop-down lists, and the skills offered and skills wanted are chosen from selection fields. Below these fields, the available days are shown as a column of check boxes beginning with Sunday.

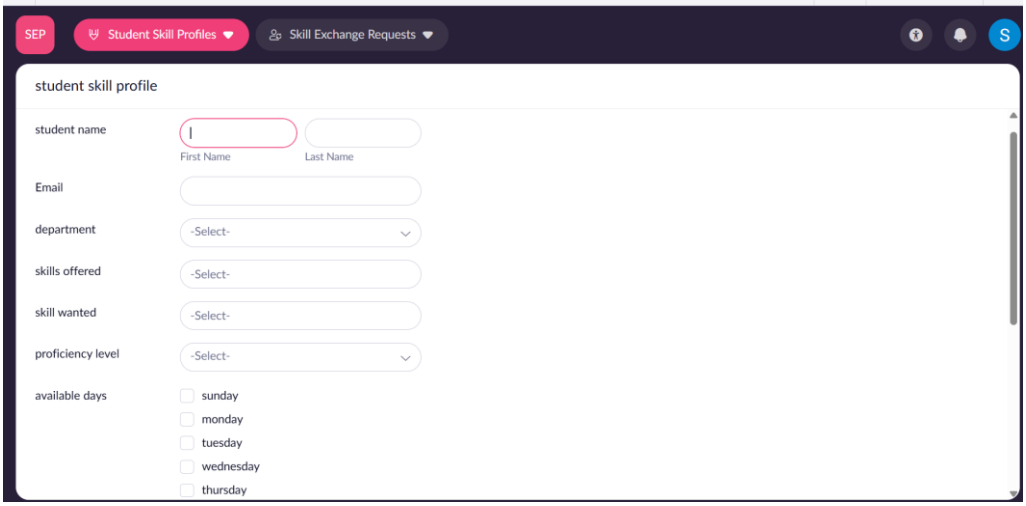


Figure 5.1: Student Skill Profile form – personal details and skills (live mode)

5.2.2 Student Skill Profile Form – Availability, Learning Mode and Certificate Upload

The lower part of the form, shown in Figure 5.2, completes the availability list through Saturday and then presents the **learning mode** drop-down, the **certificate (jpg/png)** upload control and the **summary of certificate** box. The certificate upload accepts an image of the certificate in JPG or PNG format. The summary box, which appears shaded because it is filled by the AI field rather than typed by the student, is the Certificate Summary AI field described in Section 5.7.1. A **Submit** button at the foot of the form saves the profile.

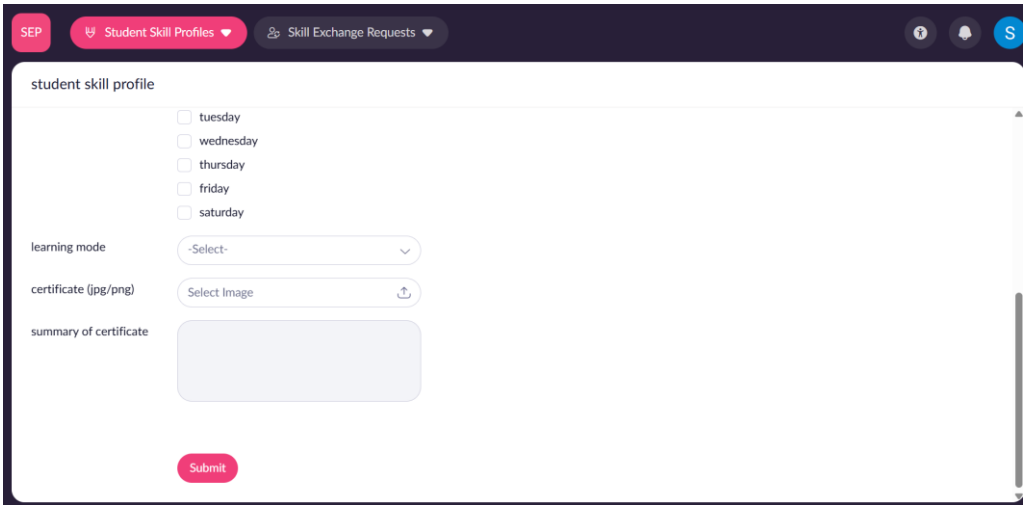


Figure 5.2: Student Skill Profile form – availability, learning mode and certificate upload

5.2.3 Skill Exchange Request Form

The request form, shown in Figure 5.3, begins with the two lookup fields **Requester** and **Learning partner**, each displayed as a drop-down list that reads its options from the Student Skill Profile form. These are followed by the selection fields **skill to learn** and **skill to teach**, the **session date** field, which expects a date in the format dd-MMM-yyyy, the **session duration** selection and the **request status** selection. A multiline **learning goal** box lets the requester describe what they hope to achieve. The fields for match score and match explanation lie further down the form.

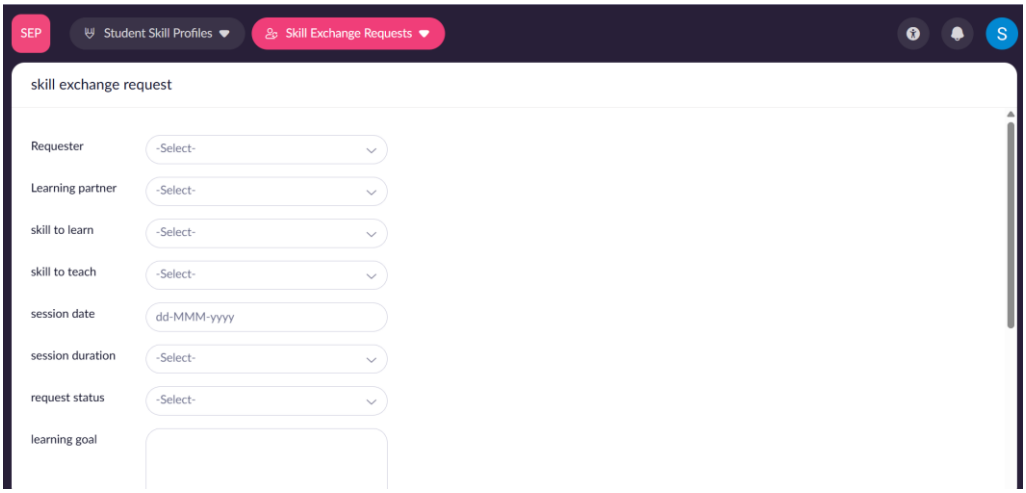


Figure 5.3: Skill Exchange Request form

5.2.4 Lookup Configuration

The Requester and Learning Partner fields were configured as lookups to the Student Skill Profile form. Both fields point to the same form, which is why both drop-down lists show the same set of students. The configuration should be confirmed in the form builder by checking that each lookup refers to Student Skill Profile and displays the student name. [Configured – Evidence]

5.3 Dataset Configuration

The coursework specifies one dataset in addition to the two forms. The project documentation from which this report is prepared does not record a separately configured dataset; the application data is held in the Student Skill Profile and Skill Exchange Request forms. The exact dataset name, source and fields must therefore be entered after the dataset configuration has been confirmed in Zoho Creator. Form records should not be described as a separate dataset unless the course instructions define them that way. [Placeholder]

Table 5.1: Dataset Configuration Attributes

Dataset Attribute	Project Details
Dataset Name	[Enter the actual dataset name]
Source / Type	[Enter the configured source or dataset type]
Fields	[List the actual dataset fields]
Purpose	[Explain how the dataset supports skill exchange]
Validation	[Describe how the dataset was tested]

5.4 Reports

The report named Skill_Exchange_Report was created to present exchange-request records. The intended columns include Requester, Learning Partner, Skill to Learn, Skill to Teach, Session Date, Request Status and Match Score. The displayed columns

should be confirmed in the application and sample records added before final evidence is captured. In addition to this report, two further views were configured: a list of all requests and a calendar of sessions.

Table 5.2: Reports Configured

Report Name	Source Form	Columns Displayed	Purpose
All Skill Exchange Requests	Skill Exchange Request	Request id, Requester, Learning partner, skill to learn, skill to teach, session date, session duration	List every stored request.
Skill_Exchange_Report	Skill Exchange Request	Requester, Learning partner, skill to learn, skill to teach, session date, request status	Consolidated review of requests and their status.
sessions (calendar)	Skill Exchange Request	Requests plotted on the session date	Show scheduled sessions by day, week or month.

5.4.1 All Skill Exchange Requests

Figure 5.4 shows the list report **All Skill Exchange Requests**, which displays every stored request in a table with a record identifier. Four sample requests are visible, with the columns Request id, Requester, Learning partner, skill to learn, skill to teach, session date and session duration. The report toolbar provides search, an add button to create a new request and a menu of further actions.

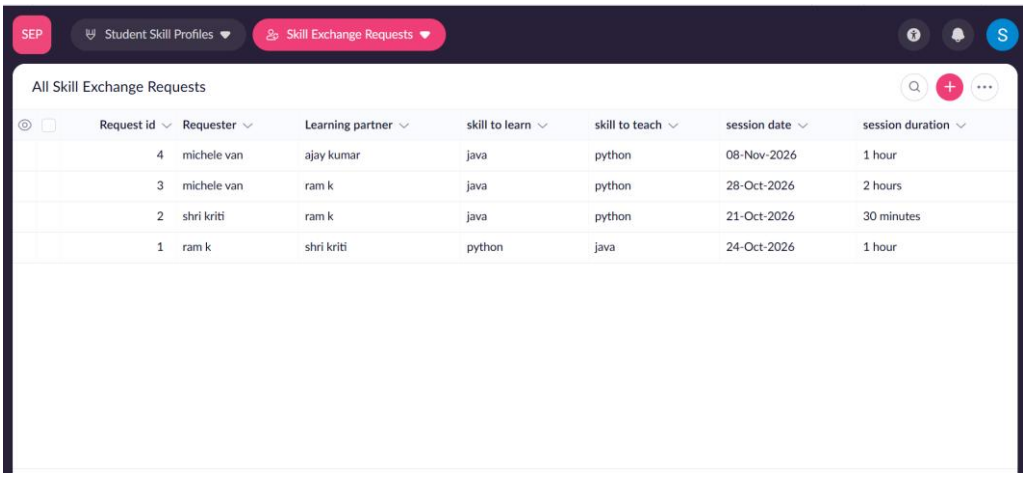


Figure 5.4: All Skill Exchange Requests report

Table 5.3 reproduces the sample records visible in Figure 5.4. These are demonstration records used during development; as noted in Chapter 7, real student data should not be added to demonstration data without permission.

Table 5.3: Sample Request Records Shown in Figure 5.4

Request id	Requester	Learning partner	Skill to learn	Skill to teach	Session date	Duration
4	michele van	ajay kumar	java	python	08-Nov-2026	1 hour

Request id	Requester	Learning partner	Skill to learn	Skill to teach	Session date	Duration
3	michele van	ram k	java	python	28-Oct-2026	2 hours
2	shri kriti	ram k	java	python	21-Oct-2026	30 minutes
1	ram k	shri kriti	python	java	24-Oct-2026	1 hour

5.4.2 Skill_Exchange_Report

Figure 5.5 shows the **skill exchange report**, a consolidated view with the columns Requester, Learning partner, skill to learn, skill to teach, session date and request status. All four sample requests display the status **accepted**, which shows that the status field is stored with each request and is available to the report. This report is the main review screen of the application: it allows a coordinator to see at a glance who is exchanging which skills, when, and in what state the request stands.

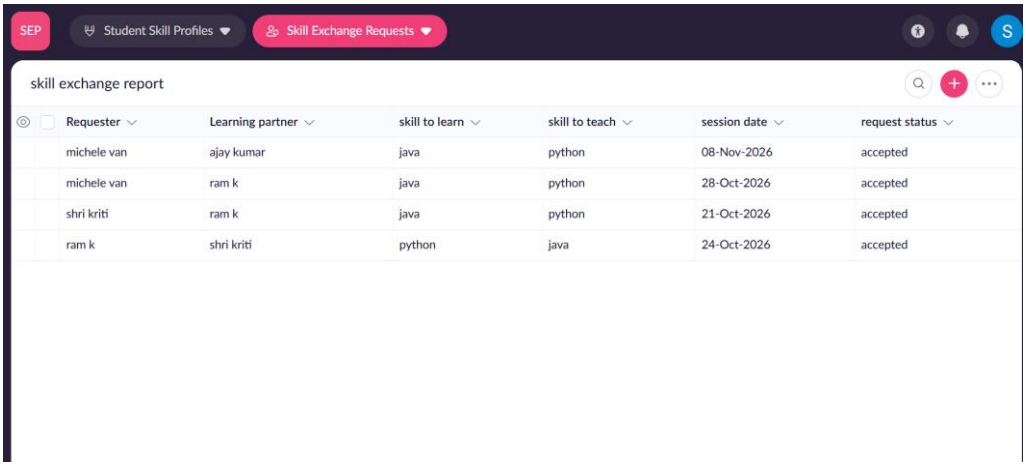


Figure 5.5: Skill_Exchange_Report showing request status

5.4.3 Calendar View of Scheduled Sessions

The third view is a calendar named **sessions**, shown in Figure 5.6 for October 2026. The calendar is based on the session date field, so each request appears as a coloured entry on its date: the entries visible in the figure fall on 21, 24 and 28 October 2026, matching the session dates of the sample requests. Month, week and day buttons allow the view to be changed, and the today button returns to the current date. A calendar is particularly helpful for students and coordinators because it presents the sessions in the same form as a personal timetable.

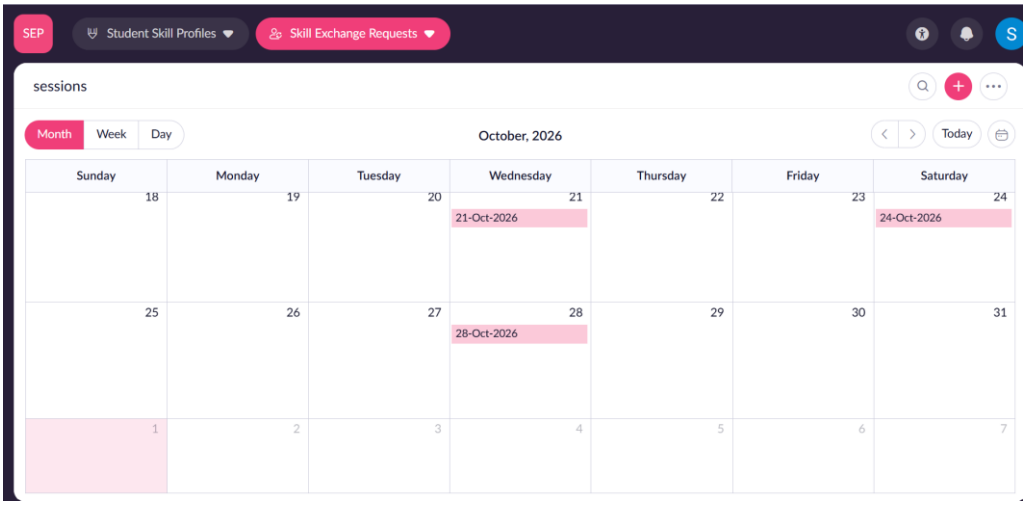


Figure 5.6: Calendar view of scheduled sessions

5.5 Deluge Scripting

Three Deluge functions were created for the application [2]. Their purpose and current description are summarised in Table 5.4. The code shown below is the code recorded in the project record and should not be treated as confirmed working code until it has been tested in the application. [Documented – Requires Verification]

Table 5.4: Deluge Functions Implemented

Function	Purpose / Current description
validate_skill_exchange	Created as a validation-related function. The final code and runtime validation should be checked.
check_session_date	Created for session-date checking. Verify the actual field link name and that past dates are rejected.
skill_match_recommendation	Created for skill-matching logic. Initial code described comparing Skills Wanted with another profile's Skills Offered; verify whether functional matching logic is saved.

All three functions exist in the application and their recorded code is reproduced in Appendix C. Their saved code, field link names and runtime behaviour are to be confirmed in the Deluge editor and by executing valid and invalid test records.

5.5.1 validate_skill_exchange

```
if(input.Requester == input.Learning_Partner)
{
    alert "Requester and Learning Partner cannot be the same student.";
    cancel submit;
}
```

This function compares the Requester field with the Learning_Partner field. Because both fields are lookups to the same profile form, the comparison is true only when the same profile record is selected in both. When that happens, the function displays the message "Requester and Learning Partner cannot be the same student." with the alert

statement and stops the save with cancel submit. The function therefore implements the self-match rule of Section 4.4.1.

5.5.2 check_session_date

```
if(input.Session_Date < zoho.currentdate)
{
    alert "Please select a valid future session date.";
    cancel submit;
}
```

This function compares the Session_Date field with zoho.currentdate, a system variable that returns the current date. If the session date is earlier than the current date, the function displays a message asking the user to choose a valid future date and cancels the submission. The function implements the date rule of Section 4.4.1. A boundary case should be tested: a session date equal to today must be accepted, since the rule is stated as “today or a future date”.

5.5.3 skill_match_recommendation

```
if(input.Skill_to_Learn == input.Skill_to_Teach)
{
    input.Match_Score = 100;
}
else
{
    input.Match_Score = 0;
}
```

The recorded code of this function compares the Skill_to_Learn and Skill_to_Teach fields and stores a Match_Score of 100 when they are equal and 0 otherwise. This is a deliberately simple placeholder for matching logic. It should be noted that in a reciprocal exchange the skill to learn and the skill to teach are normally different, as the sample records in Table 5.3 show (for example, learn java and teach python). Under the recorded code such a request would receive a score of 0. A meaningful match rule should instead compare the requested skill with the partner’s offered skills, as proposed in Section 9.2.1. The function is therefore documented as an initial version whose behaviour must be verified and improved. [Documented – Requires Verification]

5.5.4 Field Link Names

Deluge scripts read form data through field link names, which must exactly match those in the form builder. In the workflow screenshot (Figure 5.7) the date field is written in lower case as session_date, while the recorded code in Appendix C uses Session_Date. Field link names must be written exactly as the form builder defines them, so the final scripts should use whichever spelling the form builder shows. This observation is the practical reason for the maintainability requirement in Table 3.3.

5.6 Workflow Automation

The workflow named skill exchange validation1 was created under form validation / validation on submission. The workflow should be opened to confirm that it is enabled,

has the intended Deluge action, and blocks invalid submissions. A workflow name alone does not prove that its action is configured or functioning. [Documented – Requires Verification]

Figure 5.7 shows the workflow editor for skill exchange validation1. The workflow contains two blocks labelled **Execute without condition**, each holding a Deluge script. The first block tests whether the session date is earlier than the current date and, if so, displays the message "Please select today or a future session date." and cancels the submission. The second block tests whether the Requester equals the Learning partner and, if so, displays "You cannot select yourself as your learning partner." and cancels the submission. Because the blocks run without a condition, both checks are applied to every submission.

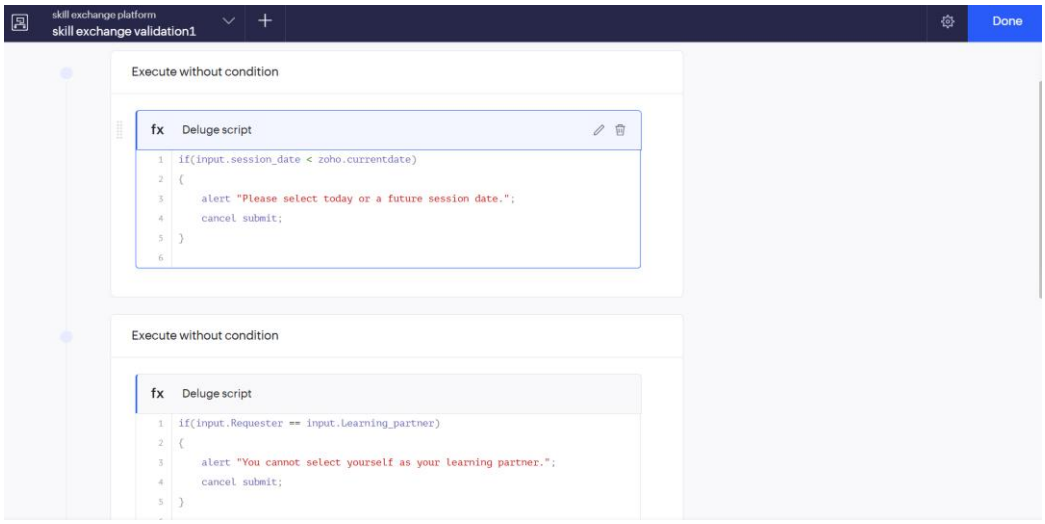


Figure 5.7: Workflow "skill exchange validation1" – Deluge validation scripts

The wording of the messages in the screenshot differs slightly from the function text recorded in Appendix C, which is natural because the workflow holds its own copy of the script. The behaviour is the same, and the two should be aligned in the final application so that a change in one place is reflected in the other. The configuration of both workflows is summarised in Table 5.5.

Table 5.5: Workflow Configuration

Workflow Attribute	**skill exchange validation1**	**skill exchange notification**
Form	Skill Exchange Request	Skill Exchange Request
Trigger	Validation on submission	Created or Edited, on successful form submission
Condition	Execute without condition (two blocks)	Execute only if request_status == "accepted"
Actions	Two Deluge scripts: reject a past session date; reject the same student as requester and partner. Each displays an alert and cancels the submission.	Send mail to zoho.loginuserid

Workflow Attribute	skill exchange validation1	skill exchange notification
Status	[Enabled / Disabled – verify]	[Enabled / Disabled – verify]
Test Evidence	[Insert test record and workflow execution screenshot]	[Insert test record and sample email]

5.7 AI and Analytics Features

5.7.1 Certificate Summary AI Field

The Student Skill Profile form includes a Certificate Summary AI field (configured as Certificate_Keywords) alongside the certificate upload. Its intended role is to summarise the certificate information provided by the student, so that a reviewer can quickly see which skills the certificate supports. [Configured – Requires Verification] The AI field and its generated summary are recorded in the feature coverage checklist (Table A.1); the summary output should be verified with an actual certificate upload before submission, and no claim is made that live AI processing works until such a test is recorded.

An AI field in Zoho Creator takes another field of the form, here the uploaded certificate image, as its input and produces a result, here a summary, that is stored in the field. In the profile form the result appears in the box labelled “summary of certificate” (Figure 5.2). The student does not type into this box. The benefit is that a coordinator reviewing a profile can read a short statement of what the certificate shows without opening the image.

5.7.2 Skill Matching

The skill_match_recommendation function provides rule-based matching, not artificial intelligence. In its recorded form it stores a score in the Match Score field by comparing the two skill fields of the request (Section 5.5.3). The AI Match Explanation field is currently described as text unless verified otherwise. A more meaningful rule, based on the overlap between Skills Wanted and Skills Offered, is proposed in Section 9.2.1.

5.7.3 Email Generation for Requests

An email-generation configuration exists for the Skill Exchange Request form: when a skill exchange request is submitted, an email to the requester is generated (Table A.1). Figure 5.8 shows the workflow “skill exchange notification” associated with email generation on accepted requests; the exact trigger recorded in the two sources differs (submission versus acceptance) and should be confirmed in the workflow editor.

The breadcrumb at the top of Figure 5.8 reads “skill exchange request › Created or Edited › Successful form submission › skill exchange notification”, which shows that the workflow belongs to the request form, runs when a record is created or edited and executes after the form has been successfully submitted. The block below it is headed

Execute only if request_status == "accepted", so the action is performed only for requests whose status is accepted. The action is **Send mail to zoho.loginuserid**, which addresses the email to the user who is logged in when the record is saved.

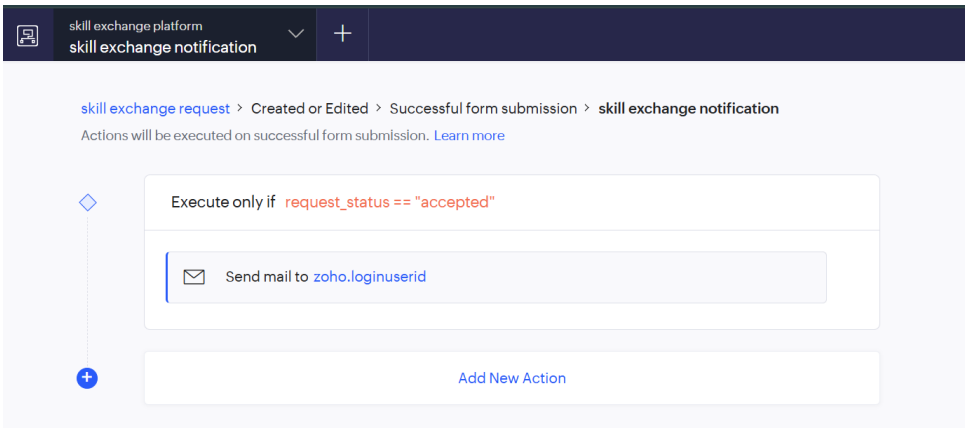


Figure 5.8: Workflow "skill exchange notification" – email on accepted requests

The following points should be confirmed before the report claims that emails are sent as intended:

- **Trigger:** the configuration shown runs on accepted requests, whereas the coverage checklist describes an email on submission. The intended behaviour should be decided and the workflow adjusted if required.
- **Recipient:** zoho.loginuserid is the email address of the logged-in user, who may be the requester or the partner. If the email must always reach the requester, the recipient should be set to the email address stored in the requester's profile.
- **Delivery:** this configuration generates an email within Zoho Creator; automatic delivery to the requester's inbox must be confirmed with a test before the report claims that emails are sent automatically. [Documented – Requires Verification]

5.8 Application Screens

The final screenshots were captured from the Zoho Creator environment and are placed in the sections where the corresponding component is explained. Table 5.6 lists each screenshot, the component it evidences and the section in which it appears, so that a reader can move easily between the explanation and the evidence.

Table 5.6: Index of Application Screenshots

Figure	Component shown	Section
Figure 5.1	Profile form – personal details and skills	5.2.1
Figure 5.2	Profile form – availability, learning mode, certificate upload and AI summary	5.2.2
Figure 5.3	Skill Exchange Request form and lookups	5.2.3
Figure 5.4	All Skill Exchange Requests report	5.4.1
Figure 5.5	Skill_Exchange_Report with request status	5.4.2

Figure	Component shown	Section
Figure 5.6	Calendar of scheduled sessions	5.4.3
Figure 5.7	Validation workflow with two Deluge scripts	5.6
Figure 5.8	Notification workflow with send-mail action	5.7.3

The following evidence should still be confirmed or captured before final submission, as it is not shown in the figures above:

- The form builder view of the Student Skill Profile and the lookup settings of the request form.
- Each of the three Deluge functions displayed in the editor, with the code saved without errors.
- A generated certificate summary for an actual certificate upload.
- A sample email produced by the notification workflow.

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing should cover both form-level data capture and the behaviour of the workflow and scripts. Table 6.1 is a test plan; each result is to be marked only after executing the test in the live application. All Actual Result entries are placeholders until the tests are executed. [Placeholder]

6.1.1 Levels of Testing

- **Form-level testing** checks that each form saves a record when valid data is entered and that required fields, drop-down lists and lookups behave as configured.
- **Script-level testing** executes each Deluge function with inputs that should pass and inputs that should be rejected, and confirms that the message displayed is the one intended.
- **Workflow-level testing** confirms that each workflow is enabled, runs on the expected event and performs the expected action.
- **Report-level testing** checks that the report and the calendar display the saved records with the correct columns and values.
- **Feature-level testing** covers the Certificate Summary AI field and the email generation, whose outputs depend on the platform's services and must be observed rather than assumed.

6.1.2 Test Data

Test data should include both valid and invalid cases. Valid data consists of at least three student profiles with different skills and at least two requests between different students. Invalid data consists of a request with a past session date, a request in which one student is both requester and partner, and a certificate upload in an unsupported format. Using fictitious names is recommended so that no real student information is exposed during testing.

6.1.3 Recording Results

For each test the tester performs the steps, observes the result and enters it in the Actual Result column together with a status of Pass or Fail. A failed test is investigated, the cause is corrected and the test is repeated. Until a test has been executed, its result remains "To verify", so the report never claims more than has been demonstrated.

6.2 Test Cases

Table 6.1 lists the planned test cases. Cases TC-01 to TC-09 cover the core features; TC-10 to TC-12 were added to cover the boundary date, the notification workflow and the calendar view.

Table 6.1: Test Cases

Test ID	Test Scenario	Expected Result	Actual Result / Status
TC-01	Save a student profile with valid required fields.	Profile record is saved.	To verify
TC-02	Create a request with valid requester and partner.	Request record is saved and appears in report.	To verify
TC-03	Select a past session date.	Submission is blocked if date validation is configured.	To verify
TC-04	Select the same student as requester and partner.	Submission is blocked if self-match validation is configured.	To verify
TC-05	Change request status to Accepted or Completed.	Status is saved and report reflects the change.	To verify
TC-06	Run the skill-match function using profiles with matching skills.	Match result or message is produced, if implemented.	To verify
TC-07	Open Skill_Exchange_Report with sample requests.	Expected columns and records are visible.	To verify
TC-08	Inspect workflow configuration.	Workflow is enabled and has the correct action.	To verify
TC-09	Upload a certificate and check the Certificate Summary AI field.	A generated summary is produced from its configured inputs.	To verify
TC-10	Select a session date equal to today.	Submission is accepted, since today is a valid date.	To verify
TC-11	Save a request with status Accepted.	The notification workflow runs and an email is generated.	To verify
TC-12	Open the sessions calendar after saving a request.	The request appears on its session date.	To verify

6.2.1 Description of the Test Cases

TC-01 and TC-02 verify the basic save operations of the two forms and the way the request appears in the report; they also confirm that the lookups list the saved profiles. **TC-03 and TC-04** are negative tests: they supply invalid input and check that the validation scripts reject it with a clear message. A negative test passes when the invalid input is refused. **TC-05** verifies that a change to the status field is stored and reflected in the report. **TC-06** exercises the matching function and, because the recorded logic is a simple comparison, its result should be interpreted in the light of the limitation discussed in Section 5.5.3. **TC-07** checks the content of the report, and **TC-08** is an inspection of the workflow in the editor rather than a data test. **TC-09** depends on the AI field and a sample certificate image. **TC-10** tests the boundary of the date rule, which

is where off-by-one errors usually occur. **TC-11** and **TC-12** cover the notification workflow and the calendar view.

6.2.2 Pass and Fail Criteria

A test passes when the observed result matches the expected result and no unexpected message or error appears. A test fails if the record is stored when it should have been refused, if a valid record is refused, if the displayed value differs from the stored value, or if a script reports an error. Tests marked “if configured” are interpreted as conditional: when the underlying feature is not configured, the test records that the feature is absent rather than that it has failed.

6.3 Debugging Deluge Scripts

A previous issue involved the session-date variable not being defined. This indicates that the Deluge code must use the exact field link name shown in the form builder. Before final submission, open each function and workflow, check field names and save the code without editor errors.

6.3.1 Common Causes of Script Errors

- **Wrong field link name:** the script refers to Session_Date while the form defines session_date, or the reverse. The editor reports that the variable is not defined.
- **Wrong event:** a validation script placed on a successful-submission event cannot cancel the submission, because the record has already been saved.
- **Wrong data type:** comparing a date field with text, or a lookup field with a name string, gives a result different from the one intended.
- **Unsaved script or disabled workflow:** the code was edited but not saved, or the workflow exists but is switched off, so the behaviour is unchanged.
- **Duplicate logic:** the same rule exists in a function and in the workflow with different wording, so a change in one place is missed in the other.

6.3.2 Debugging Approach

1. Open the form builder and note the exact field link names of all fields used in the script.
2. Open the script in the Deluge editor and read the error markers shown beside the line numbers.
3. Correct the field name or data type, save the script and reopen the workflow to confirm that it is enabled.
4. Submit one valid and one invalid record and confirm that each behaves as expected.

5. Where the cause is not obvious, temporarily display the value of a variable with an alert or info statement, then remove the statement after the check.

6.4 Defect Log

Table 6.2 records the defects and discrepancies observed during development. The status of each entry is to be updated once the correction has been made and re-tested.

Table 6.2: Defect Log

Defect ID	Observed Issue	Possible Cause	Correction / Status
BUG-01	Deluge error: the session-date variable was not defined.	Field link name in the script differs from the name in the form builder.	Use the exact link name shown in the form builder; re-test (TC-03, TC-10).
BUG-02	Date field appears as session_date in the workflow (Figure 5.7) but as Session_Date in Appendix C.	Field link names written with different spelling in two places.	Align both to the form builder spelling.
BUG-03	Validation messages in the workflow differ from the function text.	The workflow holds its own copy of each script.	Unify the wording between the functions and the workflow.
BUG-04	Match score would be 0 for reciprocal exchanges such as learn java / teach python.	Recorded rule compares Skill to Learn with Skill to Teach.	Replace with the partner-profile rule of Section 9.2.1; re-test (TC-06).
BUG-05	Notification runs for accepted requests and addresses the logged-in user, not the requester on submission.	Trigger condition and recipient differ from the intended design.	Decide the intended behaviour and adjust the workflow; re-test (TC-11).

6.5 User Acceptance Testing

User acceptance testing is carried out by asking a student to create a profile and submit a sample request, and a learning partner or administrator to review the saved request and its status. The feedback, the action taken and the final status are recorded after the demonstration.

Table 6.3: User Acceptance Feedback

User / Role	Feedback	Action Taken	Status
Student / Requester	[Enter actual feedback]	[Enter action]	Pending
Learning Partner	[Enter actual feedback]	[Enter action]	Pending
Application Administrator	[Enter actual feedback]	[Enter action]	Pending

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The application is developed in Zoho Creator and accessed through the Creator web interface. A final deployment or published URL should be recorded only if it has been configured and verified. Because Zoho Creator is a cloud platform, deployment does not involve installing software on a server; it consists of making the application available to its users and giving those users access.

7.1.1 Development and Production

A common practice is to keep a development version of an application, in which changes are made and tested, separate from the version that users rely on. Changes are moved to the live version only after they have been tested. For a student project the development version is also the demonstration version, but the principle remains valuable: test records should be removed and the final configuration reviewed before the application is shown to others.

7.1.2 Giving Users Access

Users are given access to a Zoho Creator application by inviting them and assigning them a role or permission. For this project the relevant users are students, who need to enter and view profiles and requests, and an administrator, who needs to maintain the configuration. The roles that have been actually configured, if any, should be recorded, and the access of each role should be tested with a separate account.

Because several features are marked as requiring verification, the checklist in Table 7.1 allows the final application to be confirmed in one session. Each item is checked in the live application and the result is entered in the test table.

Table 7.1: Deployment Checklist

S.No.	Deployment Checklist	Related Test	State
1	Both forms save valid records without errors.	TC-01, TC-02	To be verified
2	Requester and partner lookups show the saved profiles.	TC-02	To be verified
3	Past dates and self-matching are refused; today is accepted.	TC-03, TC-04, TC-10	To be verified
4	All three Deluge functions save without editor errors.	TC-06	To be verified
5	Both workflows are enabled and show the intended action.	TC-08, TC-11	To be verified
6	Report and calendar columns and sample records are visible.	TC-07, TC-12	To be verified
7	A summary is produced for an uploaded certificate.	TC-09	To be verified

S.No.	Deployment Checklist	Related Test	State
8	Permissions reviewed using separate accounts; application published and URL recorded, if applicable.	–	To be verified

7.2 Performance Optimisation

The application should use appropriate field types, lookup relationships and filtered reports. Lookup fields store a reference to a profile record rather than a copy of its data, so each student's details are held once. Only the fields needed for a report should be displayed. The same validation rule should not be maintained in two places with different wording (Section 6.3.1), and the improved matching function proposed in Section 9.2.1 retrieves only the learning partner's profile record instead of scanning the whole form. Completed requests may be archived or removed according to the policy of the institution so that the list views remain easy to use.

7.3 Security Features

Student names, email addresses, skill profiles, portfolio links and uploaded certificates are personal or identifying information. Access should be limited to intended users, and real student information should not be added to demonstration data without permission. Role-specific permissions should be tested if roles are configured.

7.3.1 Data Sensitivity

Table 7.2: Data Items and Their Sensitivity

Data item	Sensitivity	Recommended handling
Student name	Personal	Visible to authorised users of the application.
Email address	Personal, contact	Restrict visibility; use only for the application purpose.
Skills offered / wanted	Low	Visible to students so that matching is possible.
Portfolio URL	Low to moderate	Optional; student chooses what to share.
Certificate upload	Personal document	Restrict access; do not display publicly.
Certificate summary	Derived from document	Treat with the same care as the certificate.
Session details	Low	Visible to the two participants and the administrator.

7.3.2 Access Control Principles

- **Least privilege:** each role receives only the access it needs. A student does not need to edit another student's profile, and only the administrator needs to change forms and workflows.

- **Separation of test and real data:** fictitious data is used for testing and demonstration.
- **Review of shared data:** because both lookups display student names, the list of names seen by a user should be reviewed when roles are configured.
- **Handling of uploaded files:** certificates should be viewable only by the owner and the administrator.

7.4 Governance and Administration

The application owner should maintain consistent field link names, review workflow actions after form changes, remove test records before any public demonstration and retain screenshots of the final configuration. Backup or export features may be used if available in the account.

7.4.1 Maintenance Tasks

- After every change to a form, reopen the scripts and workflows that use its fields and confirm that they still save and run.
- Keep the wording of validation messages consistent between the functions and the workflow.
- Review the list of departments, skills and learning modes periodically so that the choice lists remain current.
- Archive or remove completed requests according to the policy of the institution.
- Retain dated screenshots of the final configuration as evidence of the state of the application.

7.4.2 Risk Overview

Table 7.3: Risks and Mitigation

Risk	Effect	Mitigation
Field name mismatch in scripts	Script errors; validation fails	Check link names in the form builder; test after each change.
Workflow disabled or incomplete	Invalid records are saved	Verify the enabled state and the action (TC-08).
Exposure of personal data	Privacy breach	Limit access; use fictitious demonstration data.
Unverified feature claimed as working	Incorrect report	Use evidence labels; record test results.
Outdated skill information	Poor matching	Encourage students to update profiles periodically.

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The application structure consists of two forms, one report, three named Deluge functions and one form workflow. The profile form captures skill-related information and certificate data; the request form records an exchange and session details; the report is intended to consolidate requests for review.

8.1.1 Consolidated Request View

The principal output of the application is the consolidated request view. Figure 8.1 shows the report as it appears with four sample requests. Each row answers the questions a coordinator would ask: who is learning, who is teaching, which skills are exchanged, when the session is planned and in what state the request stands. The same information was previously scattered across chat messages. The figure repeats the screen of Figure 5.5, now shown to discuss the result rather than to document the configuration.

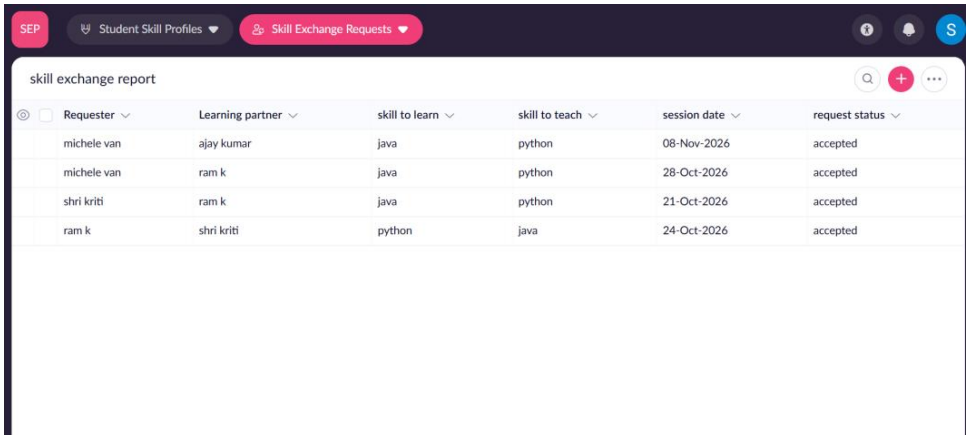


Figure 8.1: Consolidated report of exchange requests (result view; see also Figure 5.5)

8.1.2 Session Schedule

The second output is the schedule of sessions. Figure 8.2 shows the calendar for October 2026, in which accepted requests appear on their session dates. With the calendar, students and coordinators can see on which days sessions are planned, avoid clashes and prepare in time. As with the previous figure, this is the screen of Figure 5.6 shown for discussion of the result.

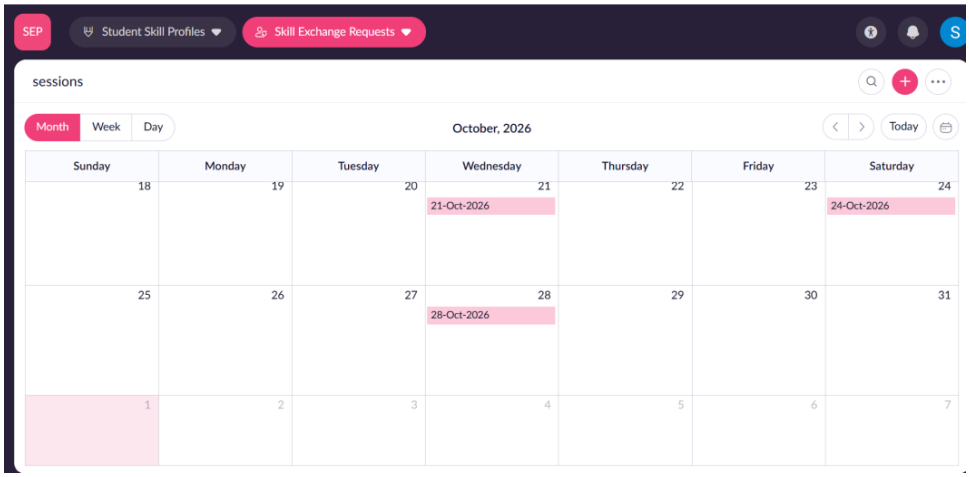


Figure 8.2: Calendar of scheduled sessions (result view; see also Figure 5.6)

8.2 Achievement of Objectives

Table 8.1 maps each objective to the component that implements it and states what remains to be checked.

Table 8.1: Mapping of Objectives to Outcomes

Objective	Implementation Component	Outcome / Evidence
Student skill directory	Student Skill Profile form	Open form and confirm fields and save operation.
Track exchange requests	Skill Exchange Request form	Create a sample request and verify it saves.
Link participants	Requester and Learning Partner lookups	Confirm both lookups point to Student Skill Profile.
Review requests	Skill_Exchange_Report	Confirm report columns and sample records.
Validation and automation	Three functions and one workflow	Inspect code and test valid/invalid inputs.
Certificate summary and email generation	Certificate Summary AI field; email generation on request submission	Verify the generated summary with a real certificate; confirm the email trigger and delivery.

8.2.1 Discussion of the Mapping

The first four objectives are supported by structures that are visible in the screenshots: the profile form (Figures 5.1 and 5.2), the request form with lookups (Figure 5.3) and the report views (Figures 5.4 to 5.6). They are therefore the most complete. The fifth objective, validation and automation, is supported by the workflow shown in Figure 5.7 and the functions of Appendix C, and awaits runtime tests. The sixth objective, certificate summary and email generation, is configured, as Figures 5.2 and 5.8 show, but depends on outputs that can be confirmed only in the live application.

The project demonstrates how a low-code platform can structure peer learning as related data records rather than informal messages. Its main value is the central profile

and request model. The next improvement is to verify the scripts end-to-end, ensure the report shows representative records, confirm the Certificate Summary AI field output and the email generation behaviour, and configure a dependable matching rule.

8.2.2 Strengths

- The data model is simple and clear: two forms, linked by lookups, from which every view is derived.
- The request form captures all details needed to arrange a session in uniform fields.
- Multiple views of the same data (list, report and calendar) were obtained by configuration alone.
- The validation rules address the two most likely input errors and are applied at the point of entry.
- The report distinguishes configured features from verified ones, which makes its claims reliable.

8.2.3 Observations from the Implementation

- The sample records show reciprocal exchanges between java and python, which confirms that the form supports the intended use case. They also show that the recorded match rule, which compares the two skill fields of a request, would not recognise such an exchange as a match.
- The validation messages in the workflow and the functions are worded differently; they should be unified.
- The notification workflow is configured to run for accepted requests and to address the logged-in user, which differs from the description of an email to the requester on submission.

8.3 Comparison with the Existing System

Table 8.2: Informal Process versus the Skill Exchange Platform

Parameter	Existing Informal Process	Proposed Application
Finding a partner	Ask in chats or among friends.	Browse profiles and choose from the lookup.
Recording a request	Message in a conversation.	Structured form with required fields.
Session details	Agreed verbally; may be forgotten.	Stored date, duration and goals.
Status	Unknown without asking.	Pending, Accepted, Rejected or Completed.
Overview for coordinators	None.	Report and calendar views.
Invalid arrangements	Possible.	Blocked by validation scripts, when verified.

8.4 Limitations of the Low-Code Approach

The application depends on the field names, form structure and workflow events configured in Zoho Creator. A change to a field link name may require a corresponding change in the Deluge scripts, and the application also depends on internet access and on the capabilities available under the selected Zoho Creator plan. The following limitations apply to the current state of the application and the report:

- The actual saved code and runtime behaviour of all functions have not been independently verified in this report.
- The workflow's action count and enabled state must be checked in the current application.
- Live AI-generated certificate summaries are not claimed without configuration and output evidence.
- Email or other notifications are not claimed unless an action is configured and successfully tested.
- Role-based access and multi-user testing require separate verification.

These limitations are stated openly so that the report reflects what has been demonstrated. Removing them requires only the verification steps described in Chapter 6, and each limitation can be closed independently as its corresponding test is completed.

8.4.1 Challenges Faced

The main challenge was the exact use of field link names in Deluge, which caused an error because a session-date variable was not defined. A second challenge was to describe AI-related features honestly: configuring an AI field is not the same as demonstrating its output, and the report therefore separates the two. A third challenge was the design of a matching rule that is meaningful for reciprocal exchanges; this remains the main area for improvement.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

The forms, field types, lookup configuration, reports and calendar view were created mainly through no-code visual configuration. The validation, matching and notification logic use Deluge scripting and workflow actions, which places those parts in the low-code category. A future requirement, such as integration with the institutional student information system or a recommendation service based on machine learning, would require external APIs or pro-code development.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Skill Exchange Platform was designed in Zoho Creator to support student-to-student learning. It includes a student profile form with certificate upload and a Certificate Summary AI field, a skill exchange request form, a consolidated report, three Deluge functions and a form workflow. Lookup fields connect request participants to profile records, while request status and session fields provide a basis for tracking the learning process, and an email to the requester is generated when a request is submitted. The application is structurally prepared, but the final report should be accompanied by live verification of the scripts, workflow actions, report data, AI field output and email delivery.

The project met its central aim of replacing scattered messages with structured records. A student can describe the skills they offer and want, a requester can select a partner from valid profiles, and the details of the session and its status are stored uniformly and displayed in reports and a calendar. The work also showed that a low-code platform can support data modelling, validation and automation within the scope of a single student project, while making clear that platform convenience does not remove the need for testing.

9.2 Future Enhancements

The enhancements below follow from the limitations identified in this report. They are listed roughly in order of importance.

9.2.1 Verified Matching Logic

Implement verified matching logic based on overlap between Skills Wanted and Skills Offered. The improved function would fetch the learning partner's profile, check whether the skill the requester wants to learn is among the skills the partner offers, check whether the skill the requester offers is among the skills the partner wants, and set the Match Score accordingly. A score of 100 could be given when both conditions hold, 50 when one holds and 0 when neither does. The outline below is illustrative only and is not part of the implemented application; it must be written against the actual field names and tested before use.

Illustrative outline (proposal, not implemented)

```
// fetch the partner profile and compare skill lists
partner = Student_Skill_Profile[ID == input.Learning_Partner];
score = 0;
if(partner.Skills_Offered.contains(input.Skill_to_Learn))
{
    score = score + 50;
}
if(partner.Skills_Wanted.contains(input.Skill_to_Teach))
{
    score = score + 50;
}
input.Match_Score = score;
```

9.2.2 Verified Certificate Summaries

Verify the Certificate Summary AI field output with real certificate uploads and, if supported, extend AI assistance to matching. Once the summaries are verified, they could also be used to suggest skills that a student may add to their profile.

9.2.3 Notifications

Confirm and extend email notifications for accepted, rejected and upcoming sessions. A useful extension is a reminder sent a day before each session, and a message to the learning partner when a new request is submitted, so that requests are not overlooked.

9.2.4 Calendar Views and Reminders

Add calendar views and reminders for session dates. The sessions calendar already shows scheduled dates; adding a personal view that shows only the sessions of the logged-in student would make it more useful.

9.2.5 Feedback and Ratings

Add student feedback and session ratings after completion. A short rating and comment would allow students to build a reputation as helpful teachers and would give coordinators information about the quality of exchanges.

9.2.6 Role-Based Access

Create role-based access and test permissions with separate accounts. Students would see their own requests and the directory of skills, while the administrator would retain full access.

9.2.7 Report Filters

Add filters to the report for status, department, skill and session date, so that a coordinator can quickly find, for example, all pending requests for a given skill.

9.3 Learning Outcomes and Career Relevance

The project provides practical experience in low-code form design, lookup relationships, report creation, Deluge scripting, workflow configuration, debugging field link names and documenting software verification honestly.

9.3.1 Technical Skills Gained

- Designing forms with suitable field types and lookup relationships.
- Configuring list reports, a consolidated report and a calendar view.
- Writing and attaching Deluge scripts for validation and calculation.
- Configuring form workflows with conditions and send-mail actions.
- Reading editor errors and correcting field link names.

9.3.2 Professional Skills Gained

The project also develops professional habits. Writing requirements before design, planning tests before declaring success and stating the limits of what has been verified are practices that apply to any software project. Documentation skills, such as writing a user guide and preparing evidence, are equally important.

9.3.3 Career Relevance

Low-code platforms are widely used by organisations that need business applications quickly, and the ability to model data, automate rules and report results on such a platform is a valuable skill. For students of Artificial Intelligence and Machine Learning, the experience is complementary: the Certificate Summary AI field shows how an AI service can be embedded in a business application, and the matching function points to how data-driven recommendations could later be added.

REFERENCES

[1] Zoho Corporation, “Zoho Creator product documentation and help resources.” [Online]. Available: https://www.zoho.com/creator/

[2] Zoho Corporation, “Zoho Creator Deluge scripting documentation.” [Online]. Available: https://www.zoho.com/deluge/help/

[3] Zoho Corporation, “Zoho Creator forms, reports and workflow documentation,” Zoho Creator Help Center.

APPENDIX A – FEATURE COVERAGE CHECKLIST

The feature coverage checklist lists every feature expected of the project, the component of the application that provides it and the status or evidence recorded for it. It is used by the examiner to confirm that the course requirements have been addressed and by the student to see which items still need evidence.

Table A.1: Feature Coverage Checklist

Feature	Project Component	Evidence / Status
Student Profile Management	Student Skill Profile form	Completed – Form screenshot
Skill Exchange Requests	Skill Exchange Request form	Completed – Form screenshot
Data Storage	Student profiles and exchange request records	Evidence – Sample records
Basic Fields	Student Name, Email Address, Department	Evidence – Field configuration
Advanced Fields	Requester and Learning Partner lookup fields	Evidence – Lookup configuration
AI Feature	Certificate Summary AI field in the form	Completed – AI field screenshot and generated summary
Email Automation	Generate an email to the requester when a skill exchange request is submitted	Completed – Email generation configuration and sample email
Deluge Script 1	validate_skill_exchange	Evidence – Script and execution result
Deluge Script 2	check_session_date	Evidence – Script and validation test
Deluge Script 3	skill_match_recommendation	Evidence – Script and execution result
Workflow Automation	skill exchange validation1	Evidence – Workflow configuration
Reports	Skill_Exchange_Report	Evidence – Report screenshot
Input Validation	Required fields and invalid-request checks	Evidence – Test results
Session Date Validation	Prevent past session dates	Evidence – Validation test
Request Status Tracking	Pending, Accepted, Rejected, Completed	Evidence – Status field screenshot
Testing	Valid and invalid request scenarios	Evidence – Test cases
Documentation	User manual, screenshots and project report	Evidence – Final report

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1 relates each learning area of the course to the sections of the report and the evidence that supports it.

Table B.1: Course Outcome Mapping

Course Learning Area	Related Report Section	Chapter	Evidence
Low-code form design (forms and fields)	4.3 Form Design; 5.2 Forms and Data Capture	Chapter 4, Chapter 5	Figures 5.1–5.3; Tables 4.2, 4.3
Data modelling and lookup relationships	4.2 Data Model	Chapter 4	Figure 4.2; Table 4.1
Reports and calendar views	5.4 Reports	Chapter 5	Figures 5.4–5.6
Deluge scripting	5.5 Deluge Scripting	Chapter 5, Appendix C	Table 5.4; Figure 5.7; Appendix C
Workflow and email automation	4.4 Process and Workflow Design; 5.6 Workflow Automation; 5.7.3 Email Generation for Requests	Chapter 4, Chapter 5	Figures 5.7, 5.8
AI-assisted features	5.7 AI and Analytics Features	Chapter 5	Certificate Summary AI field (Table A.1)
Input and date validation	5.5 Deluge Scripting; 6.3 Debugging Deluge Scripts	Chapter 5, Chapter 6	Figure 5.7; Appendix C
Testing and validation	6.1 Testing Strategy; 6.2 Test Cases; 6.4 Defect Log	Chapter 6	Tables 6.1, 6.2
Security, deployment and governance	7.1 Environments and Deployment Strategy; 7.3 Security Features; 7.4 Governance and Administration	Chapter 7	Section 7.3
Results, documentation and user manual	8.1 Application Output; 8.2 Achievement of Objectives	Chapter 8, Appendix D	Table 8.1; Appendix D

APPENDIX C – COMPLETE DELUGE SOURCE CODE

The functions are reproduced exactly as recorded in the project record. Field link names must match the form builder exactly, and the runtime behaviour of each function is to be verified in the live application. [Documented – Requires Verification]

C.1 validate_skill_exchange

Code C.1: validate_skill_exchange

```
if(input.Requester == input.Learning_Partner)
{
    alert "Requester and Learning Partner cannot be the same student.";
    cancel submit;
}
```

Explanation. The first line compares the two lookup fields. When they refer to the same profile, the block runs: the alert statement shows the message to the user and cancel submit stops the record from being saved. If the fields differ, nothing happens and the submission continues.

C.2 check_session_date

Code C.2: check_session_date

```
if(input.Session_Date < zoho.currentdate)
{
    alert "Please select a valid future session date.";
    cancel submit;
}
```

Explanation. The condition is true when the chosen session date is earlier than the system date, given by zoho.currentdate. The user is then asked to choose another date and the submission is cancelled. A date equal to today is not earlier than the current date and should therefore be accepted.

C.3 skill_match_recommendation

Code C.3: skill_match_recommendation

```
if(input.Skill_to_Learn == input.Skill_to_Teach)
{
    input.Match_Score = 100;
}
else
{
    input.Match_Score = 0;
}
```

Explanation. The function compares the two skill fields of the request and assigns a score of 100 when they are equal and 0 otherwise. As discussed in Section 5.5.3, this is a simple initial rule, and a more meaningful rule is proposed in Section 9.2.1.

Note: the scripts above are the code recorded in the project record. Confirm the field link names and the workflow trigger in the Zoho Creator editor, and paste the final tested code here before submission.

APPENDIX D – USER MANUAL

This manual explains how a student or administrator uses the application. The screens referred to are shown in Chapter 5.

D.1 Accessing the Application

1. Open the Zoho Creator application using the authorised application link provided by the administrator.
2. Sign in using the account that has been given access to the application.
3. Open the skill exchange platform application and use the two menus, Student Skill Profiles and Skill Exchange Requests, to reach the forms, reports and calendar.

D.2 Add a Student Profile

A student profile must exist before the student can take part in an exchange, because the request form chooses participants from the stored profiles. The profile form is shown in Figures 5.1 and 5.2.

1. Open the Student Skill Profile form.
2. Enter the student's name, email and department.
3. Add skills offered and skills wanted.
4. Choose proficiency level, availability and learning mode.
5. Upload a certificate if available; the Certificate Summary AI field summarises the uploaded certificate.
6. Add a portfolio URL if available, then save the record.

Tips: use the student's real name as it should appear in the lookup lists; tick every day on which the student can attend a session; keep the skills offered and wanted up to date, because they are the basis on which partners are chosen.

D.3 Create a Skill Exchange Request

A request records an agreement to exchange skills with another student. The request form is shown in Figure 5.3.

1. Open the Skill Exchange Request form.
2. Choose a Requester and a different Learning Partner from the lookup fields.
3. Enter Skill to Learn and Skill to Teach.
4. Choose a session date and duration and enter learning goals.
5. Set or retain the initial request status, then save.
6. Open Skill_Exchange_Report and confirm the request appears.

Messages you may see: “You cannot select yourself as your learning partner.” appears if the same student is chosen in both lookups; “Please select today or a future session date.” appears if the session date is in the past. In either case correct the entry and submit again.

D.4 Review and Update a Request

Use the request record to review the session details and update the status as the exchange progresses. Only describe automated notifications or match scores if they are generated by a configured and tested feature.

- Open the report (Figure 5.5) or the list of all requests (Figure 5.4) and select the request.
- Check the skills, date and duration, and read the learning goals.
- Change the request status to Accepted or Rejected after the partner responds, and to Completed after the session.
- Open the sessions calendar (Figure 5.6) to see the accepted sessions on their dates.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	SWETHA VEERAMANI
Register No.	241501261
Project Title	Skill Exchange Platform Using Zoho Creator
Date	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder pages	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		100	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.

Level	% of Max. Marks	Descriptor
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	SWETHA VEERAMANI
Register No.	241501261
Department / Section	Artificial Intelligence and Machine Learning / –
Date of Viva	To be completed
Application URL Verified	No – not verified
Demo Duration	

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	100	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER

Name: _____

EXTERNAL EXAMINER

Name: _____